# A Major Project Report on

# DeciMind AI : An AI-Powered Decision Intelligence System

Submitted in Partial fulfillment of requirements for the award of the degree of

**BACHELOR OF TECHNOLOGY**
**In**
**INFORMATION TECHNOLOGY**
**By**


**Tanmay Jain** **22BD1A12C0**


**Under the guidance of**
**Mr.C.Vikas**
**Assistant Professor, Department of IT**



**DEPARTMENT OF INFORMATION TECHNOLOGY**

**KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY**

**(AN AUTONOMOUS INSTITUTION)**
**Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH.**
**Narayanaguda, Hyderabad, Telangana-29**


**2025-26**

# KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

**(AN AUTONOMOUS INSTITUTION)**
**Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH.**
**Narayanaguda, Hyderabad, Telangana-29**
**2025-26**

## DEPARTMENT OF INFORMATION TECHNOLOGY

## CERTIFICATE

This is to certify that this is a bonafide record of the project report titled **'DeciMind AI : An AI-Powered Decision Intelligence System'**, submitted as a Major Project in IV Year II Semester (RKR21) by

**1. Tanmay Jain** **22BD1A12C0**

In partial fulfillment for the award of the degree of Bachelor of Technology in Information Technology at Keshav Memorial Institute of Technology (An Autonomous Institution) Narayanaguda, affiliated to the Jawaharlal Nehru Technological University Hyderabad.

**Faculty Supervisor**
**(Mr.C.Vikas)**

**Head of the Department**
**(Dr.G.Narender)**

Submitted for Viva Voce Examination held on _____

**External Examiner**

# Vision & Mission of Institution

## Vision:

- To be the fountainhead in producing highly skilled, globally competent engineers.
- Producing quality graduates trained in the latest software technologies and related tools and striving to make India a world leader in software products and services.

## Mission:

- To provide a learning environment that inculcates problem solving skills, professional, ethical responsibilities, lifelong learning through multi modal platforms and prepares students to become successful professionals.
- To establish an industry institute Interaction to make students ready for the industry.
- To provide exposure to students on the latest hardware and software tools.
- To promote research-based projects/activities in the emerging areas of technology convergence.
- To encourage and enable students to not merely seek jobs from the industry but also to create new enterprises.
- To induce a spirit of nationalism which will enable the student to develop, understand India's challenges and to encourage them to develop effective solutions.
- To support the faculty to accelerate their learning curve to deliver excellent service to students.

# Vision & Mission of Department

## Vision:

To produce globally competent graduates to meet the modern challenges through contemporary knowledge and moral values committed to build a vibrant nation.

## Mission:

- To create an academic environment, which promotes the intellectual and professional development of students and faculty.
- To impart skills beyond university prescribed to transform students into a well- rounded IT professional.
- To nurture the students to be dynamic, industry ready and to have multidisciplinary skills including e-learning, blended learning and remote testing as an individual and as a team.
- To continuously engage in research and projects development, strategic use of emerging technologies to attain self-sustainability.

# PROGRAM OUTCOMES (POs)

**PO1. Engineering Knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

**PO2. Problem Analysis:** Identify formulate, review research literature, and analyze complex engineering problem searching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences

**PO3. Design/Development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

**PO4. Conduct Investigations of Complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

**PO5. Modern Tool Usage:** Create select, and, apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

**PO6. The Engineer and Society:** Apply reasoning informed by contextual knowledge to societal, health, safety. Legal und cultural issues and the consequent responsibilities relevant to professional engineering practice.

**PO7. Environment and Sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts and demonstrate the knowledge of, and need for sustainable development.

**PO8. Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

**PO9. Individual and Team Work:** Function effectively as an individual, and as a member or leader in diverse teams and in multidisciplinary settings.

**PO10. Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

**PO11. Project Management and Finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

**PO12. Life-Long Learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

## PROGRAM SPECIFIC OUTCOMES (PSOs)

**PSO1:** An ability to analyze the common business functions to design and develop appropriate Information Technology solutions for social upliftments..

**PSO2:** Shall have expertise on the evolving technologies like Python, Machine Learning, Deep learning, IOT, Data Science, Full stack development, Social Networks, Cyber Security, Mobile Apps, CRM, ERP, Big Data, etc.

## PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

**PEO1:** Graduates will have successful careers in computer related engineering fields or will be able to successfully pursue advanced higher education degrees.

**PEO2:** Graduates will try and provide solutions to challenging problems in their profession by applying computer engineering principles.

**PEO3:** Graduates will engage in life-long learning and professional development by rapidly adapting to the changing work environment.

**PEO4:** Graduates will communicate effectively, work collaboratively and exhibit high levels of professionalism and ethical responsibility.

# PROJECT OUTCOMES

**P1:** Design and develop an AI Governance System that integrates rule-based decision-making with machine learning models to ensure transparency, accountability, and reliability in automated decisions.

**P2:** Implement domain-specific intelligent systems (Finance, Healthcare, HR, Cybersecurity, Education) capable of analyzing data, generating predictions, and supporting human-in-the-loop decision-making.

**P3:** Develop an analytics-driven admin dashboard to monitor AI trust score, override rate, model performance, and decision divergence in real-time.

**P4:** Enable continuous learning by training machine learning models using historical decision logs to improve system accuracy and alignment with human decisions.

## MAPPING PROJECT OUTCOMES WITH PROGRAM OUTCOMES

| PO | PO 1 | PO 2 | PO 3 | PO 4 | PO 5 | PO 6 | PO 7 | PO 8 | PO 9 | PO 10 | PO 11 | PO 1 2 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **P1** | H | M | H | | H | L | | | M | L | | |
| **P2** | M | H | | M | H | | | | | | | M |
| **P3** | M | M | H | | H | | | | M | L | | |
| **P4** | | H | | M | H | | | | | | | H |

L – LOW M –MEDIUM H– HIGH

## PROJECT OUTCOMES MAPPING WITH PROGRAM SPECIFIC OUTCOMES

| PSO | PS O 1 | PS O 2 |
|---|---|---|
| P1 | | H |
| P2 | H | M |
| P3 | M | |
| P4 | | H |

## PROJECT OUTCOMES MAPPING WITH PROGRAM EDUCATIONAL OBJECTIVES

| PEO | PE O 1 | PEO2 | PE O 3 | PE O 4 |
|---|---|---|---|---|
| P1 | M | | M | L |
| P2 | H | M | | M |
| P3 | | H | H | |
| P4 | L | M | | L |

# JUSTIFICATIONS (POs)

## P1: Design and develop an AI Governance System integrating rule-based decision-making with machine learning models

| PO | Level | Justification |
|---|---|---|
| **PO1** | **H** | Strong use of engineering fundamentals in AI governance system design |
| **PO2** | **M** | Problem analysis performed for automated decision-making |
| **PO3** | **H** | System designed using rule-based and machine learning integration |
| **PO5** | **H** | Strong use of software frameworks and intelligent system tools |
| **PO6** | **L** | Limited direct societal or sustainability impact |
| **PO9** | **M** | Project developed collaboratively across multiple modules |
| **PO10** | **L** | Basic communication through UI and reporting |

## P2: Implement domain-specific intelligent systems capable of supporting human-in-the-loop decision-making

| PO | Level | Justification |
|---|---|---|
| **PO1** | **M** | Applies intelligent computing concepts to domain-specific systems |
| **PO2** | **H** | Strong analysis of real-world domain problems |
| **PO4** | **M** | Moderate investigation using domain-based datasets |
| **PO5** | **H** | Strong use of AI tools and prediction models |
| **PO12** | **M** | Learning through implementation of adaptive systems |

## P3: Develop an analytics-driven admin dashboard to monitor AI metrics

| PO | Level | Justification |
|---|---|---|
| **PO1** | **M** | Applies software engineering concepts in dashboard design |
| **PO2** | **M** | Analytical monitoring of system performance |
| **PO3** | **H** | Strong dashboard architecture and visualization logic |
| **PO5** | **H** | Uses modern frontend frameworks and visualization libraries |
| **PO9** | **M** | Integrates multiple modules collaboratively |
| **PO10** | **L** | Supports communication through reporting and metrics |

## P4: Enable continuous learning using historical decision logs to improve AI-human alignment

| PO | Level | Justification |
|---|---|---|
| **PO2** | **H** | Strong analysis of decision trends and overrides |
| **PO4** | **M** | Moderate evaluation of learning performance |
| **PO5** | **H** | Strong use of machine learning retraining |
| **PO12** | **H** | Continuous learning and adaptive intelligence |

## JUSTIFICATIONS (PSOs)

| Project | PSO | Justification |
|---|---|---|
| **P1** | **PSO2 (H)** | Strong AI and machine learning integration |
| **P2** | **PSO1 (H)** | Strong domain-specific intelligent system development |
| **P2** | **PSO2 (M)** | Moderate automation and analytical support |
| **P3** | **PSO1 (M)** | Dashboard implementation demonstrates software skills |
| **P4** | **PSO2 (H)** | Strong adaptive learning and retraining |

## JUSTIFICATIONS (PEOs)

| Project | PEO | Level | Justification |
|---|---|---|---|
| **P1** | **PEO1** | **M** | Analytical thinking applied in system design |
| **P1** | **PEO3** | **M** | Uses modern technologies for intelligent systems |
| **P1** | **PEO4** | **L** | Limited collaboration focus |
| **P2** | **PEO1** | **H** | Strong technical problem-solving skills |
| **P2** | **PEO2** | **M** | Supports practical intelligent applications |
| **P2** | **PEO4** | **M** | Improves professional understanding of AI systems |
| **P3** | **PEO2** | **H** | Strong application of analytics and visualization |
| **P3** | **PEO3** | **H** | Effective technology integration in dashboard |
| **P4** | **PEO1** | **L** | Basic analytical contribution |
| **P4** | **PEO2** | **M** | Supports adaptive learning approach |
| **P4** | **PEO4** | **L** | Limited management and teamwork scope |

# DECLARATION

We hereby declare that the results embodied in the dissertation entitled **“DeciMind AI : An AI-Powered Decision Intelligence System"** has been carried out by us together during the academic year 2025-26 as a partial fulfillment of the award of the B.Tech degree in Information Technology from Keshav Memorial Institute of Technology (An Autonomous Institution) Narayanaguda, affiliated to JNTUH. We have not submitted this report to any other university or organization for the award of any other degree.

| Student Name | Rollno. |
| --- | --- |
| TANMAY JAIN | 22BD1A12C0 |

# ACKNOWLEDGEMENT

We take this opportunity to thank all the people who have rendered their full support to our project work. We render our thanks to **Dr. B L Malleswari**, Principal who encouraged us to do the Project.

We are grateful to **Mr. Neil Gogte**, Founder & Director, **Mr. S. Nitin**, Director for facilitating all the amenities required for carrying out this project.

We express our sincere gratitude to **Ms. Deepa Ganu**, Director Academic for providing an excellent environment in the college.

We are also thankful to **Dr.G.Narender**, Head of the Department for providing us with time to make this project a success within the given schedule.

We are also thankful to our Faculty Supervisor **Mr.C.Vikas**, for his valuable guidance and encouragement given to us throughout the project work.

We would like to thank the entire Information Technology Department faculty, who helped us directly and indirectly in the completion of the project.

We sincerely thank our friends and family for their constant motivation during the project work.

| **Student Name** | **Roll no.** |
|---|---|
| **TANMAY JAIN** | **22BD1A12C0** |

# ABSTRACT

Modern decision-making systems in domains such as finance, healthcare, human resources, cybersecurity, and education increasingly rely on automated intelligence. However, many existing AI systems lack transparency, adaptability, and alignment with human judgment. Traditional rule-based systems are often rigid and unable to generalize across changing scenarios, while standalone machine learning models frequently function as black boxes with limited interpretability and governance. To address these limitations, this project introduces the Human Decision Intelligence System (HDIS), a dual-layer AI governance framework that combines rule-based reasoning with machine learning to support transparent, adaptive, and human-aligned decision-making. The system generates baseline decisions using domain-specific rule engines and enhances them with Logistic Regression models trained on historical decision logs to improve predictive alignment with human behavior.

HDIS incorporates a human-in-the-loop architecture that allows decisions to be reviewed, overridden, and recorded, enabling continuous learning and system evolution. Explainable AI (XAI) modules provide structured reasoning by identifying the key factors influencing each decision, improving trust and interpretability. An advanced analytics dashboard monitors performance through real-time metrics such as AI Trust Score, Override Rate, AI–ML Agreement, and System Reliability. The backend is built using Flask with JWT-based authentication, while the frontend is developed using React.js to create an interactive user experience. SQLite serves as the database for storing decision logs, which also act as the training dataset for machine learning models. By integrating governance mechanisms, explainability, and feedback-driven learning, HDIS establishes a scalable and trustworthy framework for deploying AI solutions across multiple real-world domains.

# LIST OF ABBREVIATIONS

| Abbreviation | Expansion |
|---|---|
| AI | Artificial Intelligence |
| ML | Machine Learning |
| XAI | Explainable Artificial Intelligence |
| API | Application Programming Interface |
| DB | DataBase |
| UI | User Interface |
| JWT | JSON Web Token |
| DS | Decision System |
| LR | Logistic Regression |
| FLASK | Python Web Framework used for Backend Development |
| PWA | Progressive Web Application |
| RTA | Real-Time Analytics |
| SQL | Structured Query Language |

# LIST OF FIGURES

**LIST OF TABLES**

# CONTENTS

# CHAPTER 1

# 1. INTRODUCTION

## 1.1 Purpose of the Project

The rapid adoption of Artificial Intelligence across critical sectors such as finance, healthcare, human resources, cybersecurity, and education has transformed decision-making processes, enabling automation at unprecedented scale. However, this transformation has introduced a significant and growing challenge: the lack of transparency, accountability, and human alignment in automated decisions. Modern AI systems often function as opaque black boxes, producing outputs that are difficult to interpret, audit, or challenge, especially in high-stakes scenarios such as loan approvals, medical risk assessments, hiring decisions, fraud detection, and student evaluation.

The consequences of such opacity are substantial. Incorrect or biased decisions can lead to financial losses, misdiagnosis, unfair hiring practices, security vulnerabilities, and academic misclassification. Moreover, existing systems provide limited mechanisms for human oversight, making it difficult to track when and why an AI decision was overridden. This disconnect between automated intelligence and human judgment creates a critical trust deficit, hindering the deployment of AI systems in real-world governance-sensitive environments.

The Human Decision Intelligence System (HDIS) is designed to address this gap by introducing a structured AI governance framework that combines rule-based reasoning, machine learning, and human-in-the-loop validation. The system operates through three core capabilities: it generates initial decisions using domain-specific rule engines, it augments these decisions using machine learning models trained on historical patterns, and it enables human users to validate or override outcomes while recording their reasoning for future learning.

A key objective of the system is to create a continuously evolving intelligence loop, where every decision contributes to improving future predictions. By storing decision logs and training Logistic Regression models on human feedback, the system learns patterns of disagreement and adapts its behavior accordingly. This ensures that the AI system does not remain static but progressively aligns with real-world decision-making practices.To further enhance trust and interpretability, the system integrates Explainable AI (XAI) techniques that provide clear, structured explanations for each decision. Users can understand not only what decision was made

but also why it was made, based on contributing features and reasoning factors. Additionally, an advanced administrative dashboard offers real-time insights into system performance through metrics such as AI Trust Score, Override Rate, AI–ML Agreement, and System Reliability, enabling continuous monitoring and governance.

Ultimately, this project serves as both a functional AI governance platform and a demonstration of how combining rule-based systems, machine learning, and human feedback can produce transparent, accountable, and adaptive intelligent systems. It highlights the importance of bridging the gap between automated decision-making and human judgment, paving the way for more trustworthy and deployable AI solutions in multi-domain environments.

## 1.2 Problems with Existing Systems

Most modern AI systems operate as black-box models, particularly those based on machine learning and deep learning. These systems generate predictions without providing interpretable reasoning behind their outputs. In domains such as healthcare risk assessment or loan approval, this lack of transparency makes it difficult for users to understand why a decision was made, reducing trust and limiting practical usability. Existing platforms do not provide structured explanations or feature-level reasoning, making auditing and validation nearly impossible.

### 1.2.1 Lack of Transparency in AI Decision-Making

Google Maps, Apple Maps, HERE, and the MapMyIndia consumer navigation apps provide route planning, traffic-aware guidance, and turn-by-turn navigation. However, none of them model road surface quality at the micro level. These systems treat every navigable road segment as a uniform surface, routing users over severely potholed roads as readily as over freshly resurfaced ones. The underlying data models used by these applications — OpenStreetMap, TomTom, HERE HD Maps, or proprietary data — contain road segment topology, connectivity, and classification, but do not contain pothole positions, surface condition scores, or road damage records. The omission is structural: these platforms were not architected to ingest, store, or surface sub-metre road condition events.

### 1.2.2 Absence of Human-in-the-Loop Governance

Conventional AI systems are designed for full automation, with minimal or no human intervention. While this improves efficiency, it introduces significant risks when incorrect

decisions are made. There is no standardized mechanism to allow humans to review, override, or provide feedback on AI decisions. As a result, systems cannot learn from real-world corrections, leading to repeated errors and misalignment with human judgment.

### 1.2.3 Static Rule-Based Systems Lack Adaptability

Traditional decision systems based on predefined rules are widely used in industries such as finance and HR. However, these systems are rigid and fail to adapt to evolving data patterns. Once deployed, rule-based systems require manual updates to remain relevant, making them unsuitable for dynamic environments where decision criteria continuously change. They also fail to capture complex relationships in data, limiting their predictive capability.

### 1.2.4 Machine Learning Models Lack Governance and Monitoring

While machine learning models provide improved predictive accuracy, they often operate without governance frameworks. There is no built-in mechanism to monitor model performance, detect divergence from human decisions, or evaluate trust levels over time. Metrics such as override rate, decision consistency, and system reliability are typically not tracked, making it difficult to assess whether the model is improving or degrading.

### 1.2.5 Lack of Continuous Learning from Real-World Feedback

Most deployed AI systems are trained once and remain static unless retrained manually. They do not utilize real-time decision logs or human feedback to improve performance. This results in models that become outdated over time and fail to adapt to new patterns or edge cases. The absence of feedback-driven learning prevents the system from evolving into a more accurate and human-aligned intelligence.

### 1.2.6 No Unified Multi-Domain Decision Platform and Limited Explainability and Visualization Tools

Existing solutions are typically domain-specific, focusing on a single application such as fraud detection, recruitment screening, or medical diagnosis. There is no integrated platform that can handle multiple domains under a unified governance framework. This leads to fragmented systems with inconsistent logic, duplicated efforts, and lack of standardization in decision-making processes across domains.

Most systems do not provide visual insights into how decisions are made or how models perform over time. The absence of dashboards and analytics tools prevents administrators from monitoring key metrics such as AI Trust Score, AI–ML agreement, and override trends. Without such visibility, decision systems operate without accountability or measurable performance indicators.

## 1.3 Proposed System

The Human Decision Intelligence System (HDIS) proposes a four-layer AI governance architecture that addresses the limitations of existing decision systems by combining rule-based logic, machine learning, explainability, and real-time analytics into a unified framework.

**Layer 1: Data Acquisition and Decision Logging Layer**

The foundation of the system is a structured data layer that captures user inputs, decision parameters, and historical decision logs across multiple domains including Finance, Healthcare, Human Resources, Cybersecurity, and Education. Each decision instance is stored with attributes such as input features, AI-generated decision, confidence score, human override (if any), reasoning, and timestamp.

This layer is implemented using a lightweight relational database (SQLite), enabling efficient storage and retrieval of decision records. The decision logs serve a dual purpose: they act as an audit trail for governance and as a continuously growing dataset for training machine learning models.

**Layer 2: Dual-Mode Decision Engine (Rule-Based + Machine Learning)**

The system employs a hybrid decision-making approach through two complementary components:

•Rule-Based Engine:

Domain-specific rules are defined to generate baseline decisions using deterministic logic. These rules ensure consistency, interpretability, and immediate response for standard cases.

•Machine Learning Model (Logistic Regression):

A supervised learning model is trained on historical decision logs to capture patterns in human overrides and improve predictive accuracy. The model processes input features and outputs a secondary decision aligned with learned behavior.

This dual-mode pipeline enables comparison between rule-based decisions and ML predictions, forming the basis for governance analysis and system improvement.

**Layer 3: Explainable AI (XAI) and Human-in-the-Loop Interface**

To ensure transparency and trust, the system integrates Explainable AI modules that generate structured explanations for each decision. These explanations include feature-level contributions and reasoning summaries, allowing users to understand why a particular decision was made.

A human-in-the-loop interface enables users to review AI decisions, provide overrides when necessary, and submit reasoning. This interaction is critical for maintaining accountability and creating a feedback loop that enhances model learning over time.

**Layer 4: Admin Dashboard and Governance Analytics Layer**

The administrative dashboard serves as the central monitoring and analytics hub of the system. Built using React.js, it provides real-time visualization of key governance metrics including:

• AI Trust Score (AI vs Human agreement)

• Override Rate (frequency of human intervention)

• AI–ML Agreement (model consistency)

• System Reliability Index

The dashboard also includes a divergence analysis engine that evaluates decision discrepancies across different domains, enabling administrators to identify weak areas and improve system performance.

## 1.4 Scope of the Project

The scope of the Human Decision Intelligence System (HDIS) is defined across operational, technical, and exclusion dimensions, outlining the boundaries and capabilities of the proposed system.

### 1.4.1 Operational Scope

The system is designed to operate across multiple decision-making domains, including Finance, Healthcare, Human Resources, Cybersecurity, and Education. It supports scenarios such as loan approval prediction, health risk assessment, candidate evaluation, fraud detection, and student performance analysis.The system enables users to input relevant parameters, receive AI-generated decisions, and optionally override them with reasoning. It supports real-time interaction through a web-based interface and allows continuous monitoring of decisions through an administrative dashboard. The platform is designed for moderate-scale usage, supporting multiple users and continuous decision logging during active sessions.

### 1.4.2 Technical Scope

The technical scope includes the complete development and integration of all software components required for the system. The backend is implemented using Flask, providing RESTful APIs for decision processing, authentication, and data management. The frontend is developed using React.js, delivering an interactive and responsive user interface. The system incorporates rule-based decision engines and machine learning models (Logistic Regression) trained on historical decision logs. It includes Explainable AI (XAI) modules for generating interpretable decision explanations. The database is implemented using SQLite, storing user data and decision logs used for analytics and model training. The scope also covers the development of an admin dashboard with real-time analytics, including metrics such as AI Trust Score, Override Rate, AI–ML Agreement, and System Reliability. Additionally, it includes model training scripts, data preprocessing, and integration of trained models into the backend inference pipeline.

### 1.4.3 Exclusions

The following aspects are beyond the current scope of the project but may be considered for future enhancements:

• Integration of advanced deep learning models (e.g., neural networks) for higher prediction accuracy

• Deployment on large-scale distributed cloud infrastructure for enterprise-level scalability

• Real-time streaming analytics using technologies such as WebSockets or Kafka

• Automated bias detection and fairness auditing mechanisms

• Integration with external enterprise systems or government regulatory APIs

• Support for multilingual interfaces and mobile application deployment

## 1.5 System Architecture Overview

The complete system architecture of the Human Decision Intelligence System (HDIS) consists of multiple interconnected layers that enable intelligent, transparent, and human-governed decision-making.

- **Presentation Layer:** The system begins with a React-based user interface where users provide input data and receive outputs in the form of AI decisions and explainable insights (XAI). This layer ensures user interaction and visualization of results.
- **Application Layer:** User inputs are transmitted to a Flask-based API server that handles authentication, routing, and communication between system components. This layer acts as the central control unit of the system.
- **Decision Layer:** EThe core intelligence of the system is divided into three subsystems:
  The Rule-Based Engine generates deterministic decisions using predefined logic.
  The Machine Learning Engine applies Logistic Regression models to produce predictive decisions.
  The Explainable AI (XAI) module provides feature-level reasoning behind decisions.
- **Decision Fusion Layer:** Outputs from the rule-based and ML engines are combined and compared to generate final decision insights along with confidence scores. This layer ensures consistency and enables AI–ML comparison.
- **Human-in-the-Loop Layer:**Users are given the ability to accept or override AI decisions. Any override is recorded along with reasoning, enabling accountability and continuous system improvement.
- **Data Layer:** All decisions, overrides, and logs are stored in a SQLite database. This data serves as both an audit trail and a training dataset for improving ML models.
- **Analytics Layer:** The admin dashboard visualizes system performance using metrics such as AI Trust Score, Override Rate, ML Agreement, and system reliability, enabling real-time governance and monitoring.

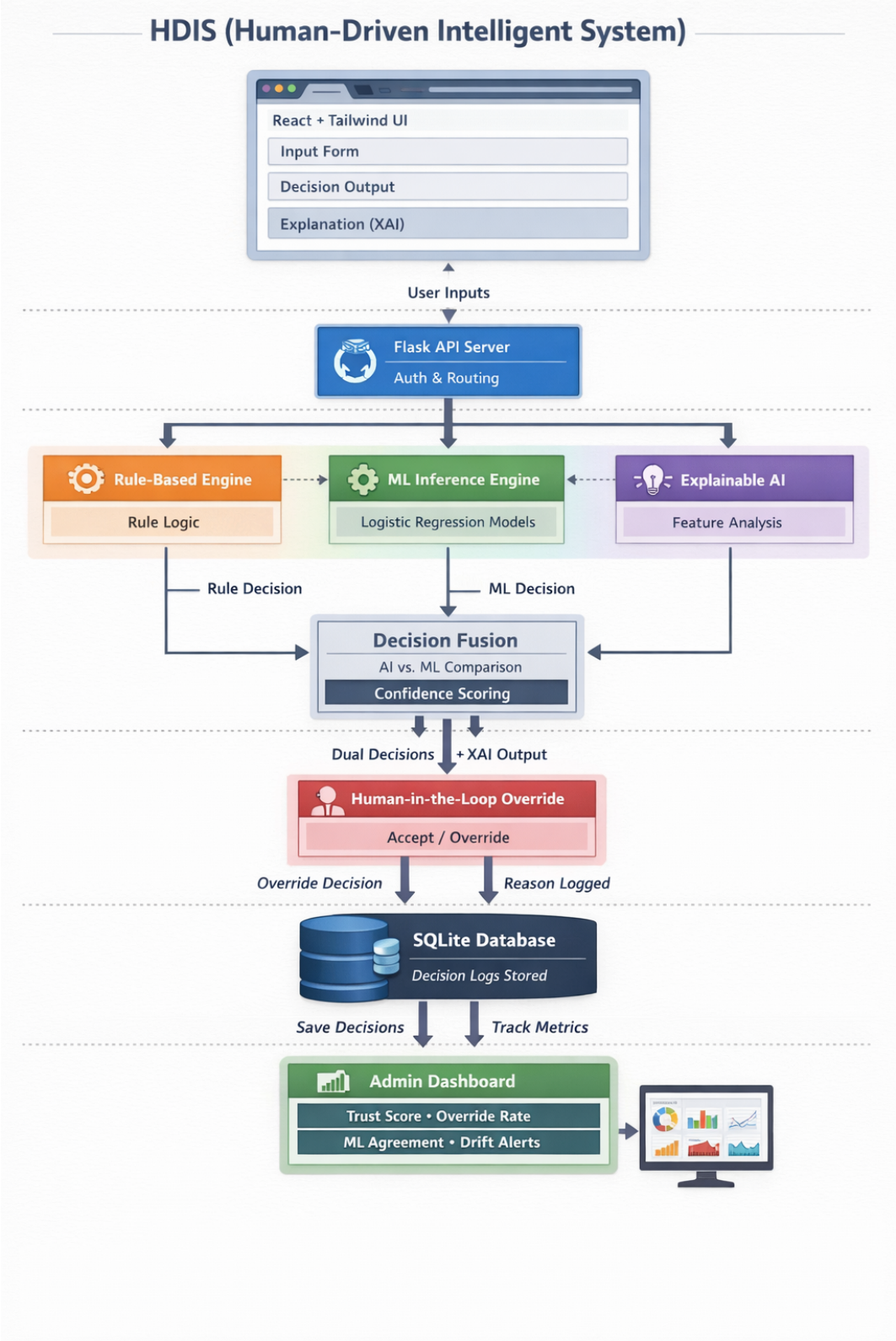



***Figure 1.1:*** *System Architecture – Data Flow from User Input to AI Governance Dashboard in HDIS*

\

# CHAPTER 2

# 2. LITERATURE SURVEY

## 2.1 Evolution of Intelligent Decision-Making Systems

Decision-making systems have evolved significantly over the past several decades, transitioning from simple rule-based frameworks to complex, data-driven artificial intelligence systems. Early decision support systems (DSS), developed during the 1960s and 1970s, were primarily based on deterministic logic and statistical models. These systems relied on predefined rules and structured data to assist human decision-makers in domains such as finance, operations, and healthcare. While effective for well-defined problems, they lacked adaptability and were unable to handle uncertainty or dynamic data environments.

The 1980s and 1990s saw the emergence of expert systems, which attempted to replicate human expertise using knowledge bases and inference engines. Systems such as MYCIN in healthcare demonstrated the potential of rule-based reasoning in complex domains. However, these systems required extensive manual knowledge engineering and were difficult to scale or update, limiting their widespread adoption.

With the rise of machine learning in the early 2000s, decision-making systems began shifting towards data-driven approaches. Algorithms such as decision trees, support vector machines, and logistic regression enabled systems to learn patterns from historical data rather than relying solely on predefined rules. This transition significantly improved predictive accuracy and allowed systems to handle more complex and unstructured problems. However, these models introduced a new challenge: lack of interpretability. As models became more sophisticated, understanding how decisions were made became increasingly difficult.

The period from 2010 onwards marked the dominance of deep learning and large-scale AI systems. Neural networks achieved remarkable success in fields such as image recognition, natural language processing, and predictive analytics. Despite their performance advantages, these systems often functioned as black boxes, providing little to no explanation for their outputs. This lack of transparency raised serious concerns in high-stakes domains, leading to regulatory and ethical challenges.

In response to these limitations, the concept of Explainable Artificial Intelligence (XAI) emerged as a critical research area. Techniques such as feature importance analysis, SHAP values, and interpretable models were developed to provide insights into model behavior. Simultaneously, the idea of Human-in-the-Loop (HITL) systems gained traction, emphasizing the importance of incorporating human judgment into automated decision processes to ensure accountability and reliability.More recently, research has focused on AI governance frameworks that combine rule-based logic, machine learning, and human feedback into unified systems. These hybrid approaches aim to balance accuracy with interpretability while enabling continuous learning from real-world interactions. Modern systems now integrate analytics dashboards, monitoring tools, and feedback mechanisms to track performance metrics such as trust, reliability, and decision consistency.

The current generation of intelligent systems, including the proposed Human Decision Intelligence System (HDIS), represents a convergence of these advancements. By integrating rule-based engines, machine learning models, explainability modules, and governance analytics into a single architecture, such systems address the critical limitations of previous approaches and provide a scalable, transparent, and adaptive solution for real-world decision-making across multiple domains.

## 2.2 Decision Intelligence and Explainability in AI Systems

The effectiveness of modern AI-driven decision systems depends on how accurately they process input features, generate predictions, and align with human reasoning. Unlike physical signal processing systems, decision intelligence operates on structured and semi-structured data where patterns are learned from historical behavior. The challenge lies not only in generating accurate predictions but also in ensuring interpretability, consistency, and adaptability across diverse domains.

### 2.2.1 Classical Rule-Based Decision Systems

Early decision-making systems relied heavily on rule-based logic, where decisions were derived from predefined conditions and thresholds. For example, in financial systems, a loan might be approved if income exceeds a certain value and credit score crosses a fixed threshold. These systems are computationally efficient and highly interpretable, making them suitable for regulated environments.However, rule-based systems suffer from significant limitations. They are static in nature and fail to adapt to changing patterns in data. Complex real-world scenarios often involve non-linear relationships between variables, which cannot be effectively captured through fixed rules. Additionally, maintaining and updating rule sets becomes increasingly difficult as system complexity grows, leading to inconsistencies and outdated logic.

### 2.2.2 Statistical and Machine Learning Approaches

To overcome the rigidity of rule-based systems, machine learning techniques such as Logistic Regression, Decision Trees, and Support Vector Machines were introduced. These models learn patterns from historical data and generate probabilistic predictions based on input features. Logistic Regression, widely used in classification problems, models the relationship between input variables and the probability of a particular outcome. It provides a balance between interpretability and predictive capability, making it suitable for domains like healthcare risk assessment and financial decision-making.

Feature scaling, normalization, and transformation techniques play a crucial role in improving model performance. By standardizing input variables, models can better capture relationships between features and outcomes. However, traditional ML models still depend on the quality and quantity of training data and may struggle with unseen or highly complex scenarios.

### 2.2.3 The Need for Explainable AI (XAI)

As AI systems became more complex, particularly with the rise of data-driven models, the lack of transparency emerged as a critical issue. Explainable AI (XAI) addresses this challenge by providing insights into how models arrive at specific decisions. Techniques such as feature importance analysis and contribution scoring allow systems to highlight which input variables had the most influence on a decision. This is particularly important in high-stakes domains where users must justify decisions, such as rejecting a loan or classifying a patient as high risk.

Explainability not only improves user trust but also enables debugging and validation of models. It ensures that decisions are not based on biased or irrelevant features, thereby enhancing fairness and accountability.

### 2.2.4 Human-in-the-Loop Decision Systems

A key advancement in modern AI systems is the integration of Human-in-the-Loop (HITL) mechanisms. In this approach, AI-generated decisions are not considered final; instead, they are reviewed and, if necessary, overridden by human experts. This interaction creates a feedback loop where human corrections are recorded and used to improve future model performance. It also ensures that critical decisions remain aligned with human judgment, reducing the risk of automated errors. However, implementing HITL systems requires careful design to balance automation and human intervention. Excessive reliance on human input can reduce system efficiency, while insufficient oversight can lead to poor decision quality.

## 2.3 Machine Learning and Explainable AI for Decision Systems

### 2.3.1 Classical Machine Learning Approaches — Logistic Regression and Beyond

The adoption of machine learning in decision systems began with statistical models such as Logistic Regression, Naïve Bayes, and Decision Trees. Among these, Logistic Regression has remained one of the most widely used techniques for binary classification problems due to its simplicity, interpretability, and strong baseline performance. Logistic Regression models the probability of an outcome using a sigmoid function applied to a linear combination of input features. This makes it particularly suitable for domains such as loan approval, risk assessment, and classification tasks where the relationship between variables can be approximated linearly. Unlike complex models, Logistic Regression provides direct insight into feature influence through its coefficients, making it easier to interpret and validate.

However, classical machine learning models are limited in their ability to capture complex, non-linear relationships in high-dimensional data. Their performance heavily depends on feature engineering and data quality, and they may struggle in scenarios with intricate decision boundaries or noisy datasets.

### 2.3.2 Explainable AI for Transparent Decision-Making

As machine learning models became increasingly integrated into critical systems, the need for transparency led to the development of Explainable Artificial Intelligence (XAI). XAI techniques aim to provide meaningful insights into how input features influence model predictions.

Feature importance methods, coefficient analysis, and contribution scoring are commonly used to interpret models such as Logistic Regression. These techniques allow systems to generate explanations in terms of positive and negative impacts of each feature on the final decision. Such explanations are essential in domains like healthcare and finance, where decisions must be justified to users and regulatory bodies.

Explainability also plays a crucial role in identifying biases and ensuring fairness. By analyzing which features dominate decision-making, developers can detect unintended correlations and improve model robustness.

### 2.3.3 Hybrid Decision Systems — Combining Rule-Based and ML Models

Modern intelligent systems increasingly adopt hybrid architectures that combine rule-based logic with machine learning models. Rule-based systems provide consistency and interpretability, while machine learning models offer adaptability and improved predictive accuracy.

This combination enables systems to generate multiple decision perspectives: a deterministic output from rules and a probabilistic output from ML models. Comparing these outputs allows the system to detect inconsistencies and measure agreement between different decision mechanisms.

Such hybrid systems are particularly effective in governance-driven environments, where both accuracy and explainability are required. They also provide a fallback mechanism, ensuring that decisions remain reliable even when one component underperforms.

#### 2.3.4 Human-in-the-Loop Learning and Feedback Integration

A significant advancement in intelligent decision systems is the integration of Human-in-the-Loop (HITL) mechanisms. In this approach, human users actively participate in validating AI-generated decisions by either accepting or overriding them.

Human feedback serves as a valuable source of labeled data, enabling continuous learning and model improvement. By incorporating override decisions into training datasets, the system can adapt to real-world scenarios and align more closely with human judgment.

This feedback-driven learning process transforms static models into adaptive systems capable of evolving over time. It also enhances trust and accountability, as users retain control over final decisions.

### 2.3.5 Decision Analytics and Governance Metrics

The effectiveness of AI systems cannot be evaluated solely based on prediction accuracy. Modern systems require governance metrics to assess reliability, consistency, and alignment with human decisions.

Metrics such as AI Trust Score (agreement between AI and human decisions), Override Rate (frequency of human intervention), and AI–ML Agreement (consistency between different models) provide valuable insights into system performance. These metrics enable administrators to monitor system behavior, identify weaknesses, and take corrective actions.

Visualization tools and dashboards play a crucial role in presenting these metrics, transforming raw data into actionable insights for decision-makers.

### 2.3.6 Continuous Model Improvement through Data-Driven Learning

One of the defining features of modern AI systems is their ability to improve over time through continuous learning. Decision logs, which capture inputs, predictions, overrides, and reasoning, serve as a dynamic dataset for retraining models.

By periodically updating models with new data, systems can adapt to changing patterns and improve their predictive capabilities. This iterative learning process ensures long-term system relevance and performance stability.

## 2.4 Explainable and Interpretable AI Architectures — Detailed Analysis

### 2.4.1 Interpretable Machine Learning Models

Early approaches to interpretable AI focused on inherently transparent models such as Logistic Regression, Decision Trees, and Linear Models. These models provide direct insight into decision-making by expressing outcomes as mathematical relationships between input features and outputs. Logistic Regression, in particular, models the probability of an outcome using a weighted combination of features. The coefficients associated with each feature indicate its influence on the final decision, making it highly suitable for domains requiring explainability. Unlike black-box models, these approaches allow direct auditing and validation of decisions, which is critical in governance-driven systems. However, their limitation lies in their inability to capture complex non-linear relationships, reducing performance in highly dynamic environments.

### 2.4.2 Post-Hoc Explainability Techniques

To address the limitations of black-box models, post-hoc explainability techniques were developed. These methods generate explanations after a model has made a prediction, without altering the model itself. Popular approaches include feature importance scoring, local explanation methods, and contribution analysis. These techniques estimate how much each input feature contributes to a particular prediction, allowing users to interpret model behavior even when the underlying model is complex. While post-hoc methods improve interpretability, they may introduce approximation errors and do not always fully reflect the internal reasoning of the model. This creates a trade-off between accuracy and interpretability.

### 2.4.3 Hybrid Explainable Architectures

Modern AI systems increasingly adopt hybrid architectures that combine inherently interpretable models with explainability techniques. In such systems, rule-based logic provides transparent baseline decisions, while machine learning models enhance predictive performance. This dual-architecture approach allows systems to maintain interpretability while benefiting from data-driven learning. The outputs of both components can be compared to detect inconsistencies, enabling better governance and reliability. Hybrid architectures also support layered explainability, where decisions can be explained at both the rule level (logical reasoning) and the model level (feature contribution).

## 2.5 Learning from Human Feedback and Adaptive Decision Systems

Modern intelligent systems increasingly rely on adaptive learning techniques that leverage real-world interactions to improve performance over time. Unlike traditional supervised learning, where models are trained on static labeled datasets, adaptive systems utilize continuous streams of data generated during system usage. In decision intelligence systems, this data primarily consists of user inputs, AI-generated decisions, and human overrides, forming a rich source of implicit supervision.

This paradigm is particularly valuable because labeled decision data is inherently expensive and context-dependent, while operational logs are continuously generated during system use. By treating human feedback as a supervisory signal, systems can evolve dynamically and align more closely with real-world decision-making patterns.

### 2.5.1 Learning from Human Feedback (Human-in-the-Loop Learning)

Human-in-the-Loop (HITL) learning is a framework where human decisions are incorporated into the training process to improve model performance. In this approach, AI-generated decisions are reviewed by users, and any corrections or overrides are recorded as labeled data. This feedback mechanism enables the system to learn from mistakes and adapt to domain-specific nuances that may not be captured during initial training. Over time, the model becomes more aligned with human judgment, reducing the need for frequent overrides. HITL learning also enhances accountability, as every decision can be traced back to both AI reasoning and human validation. This is particularly important in high-stakes domains such as healthcare and finance, where incorrect decisions can have significant consequences.

### 2.5.2 Feature Importance and Adaptive Weighting

In adaptive decision systems, understanding the importance of input features is critical for both performance and interpretability. Techniques such as coefficient analysis in Logistic Regression allow the system to assign weights to features based on their contribution to the outcome. As new data is incorporated through feedback, these weights are updated, enabling the model to adjust its decision boundaries dynamically. This process acts as an implicit feature selection mechanism, where more relevant features gain influence while less relevant ones are suppressed.

Such adaptive weighting ensures that the system remains responsive to changing patterns in data, improving both accuracy and robustness over time.

## 2.6 Integrated AI Governance and Decision Monitoring Systems

The integration of intelligent decision systems with real-time monitoring and analytics platforms has emerged as a critical area of research under AI governance and responsible AI frameworks. Modern systems are no longer evaluated solely on prediction accuracy but on their ability to provide transparency, accountability, and continuous performance monitoring.

The concept of Human-Centered AI and AI Governance emphasizes that intelligent systems must operate within a feedback-driven ecosystem where decisions are logged, monitored, and continuously improved. Similar to Volunteered Geographic Information (VGI) systems, decision intelligence platforms rely on continuous data generation from user interactions, enabling real-time system evolution. In this context, decision logs act as dynamic data streams that capture system behavior across multiple domains.

The Human Decision Intelligence System (HDIS) is designed around these principles by integrating decision-making, feedback collection, and governance analytics into a unified architecture. The system ensures that every decision contributes to a continuously improving knowledge base, enabling adaptive and transparent intelligence.

### 2.6.1 Real-Time Analytics and Dashboard Systems

Modern AI systems increasingly incorporate real-time dashboards to monitor performance metrics and system behavior. These dashboards provide insights into key governance indicators such as decision consistency, override patterns, and model reliability.

The admin dashboard in HDIS is designed to function as a real-time monitoring system, enabling administrators to track metrics such as AI Trust Score, Override Rate, AI–ML Agreement, and System Reliability. Unlike traditional systems that rely on static reports, real-time analytics allows immediate detection of anomalies, model drift, and inconsistencies across domains.

Such dashboards transform raw decision data into actionable insights, enabling informed decision-making and system optimization.

### 2.6.2 Database Systems for Decision Intelligence

Efficient data storage and retrieval are essential for maintaining decision logs and supporting continuous learning. Lightweight relational databases such as SQLite provide a practical solution for managing structured decision data in small to medium-scale systems.

SQLite enables fast querying, low overhead, and easy integration with backend frameworks, making it suitable for storing user inputs, AI decisions, human overrides, and reasoning logs. This structured data forms the foundation for analytics, auditing, and machine learning model training.

Additionally, maintaining a centralized decision log ensures traceability and accountability, which are critical requirements in governance-driven systems.

## 2.7 Research Gap and Novel Contributions

The literature survey demonstrates that while significant advancements have been made in rule-based systems, machine learning models, explainable AI, and human-in-the-loop frameworks, existing solutions fail to integrate these components into a unified, end-to-end decision intelligence platform. Most systems address individual aspects such as prediction accuracy, interpretability, or automation in isolation, but do not provide a comprehensive governance-driven architecture.

The following key research gaps have been identified:

• Lack of Integrated Hybrid Decision Systems: Existing systems typically rely either on rule-based logic or machine learning models, but not both simultaneously. There is limited work on combining deterministic reasoning with data-driven learning in a single framework.

• Absence of Human-in-the-Loop Learning Pipelines: While human feedback is recognized as important, most systems do not incorporate structured mechanisms to capture, store, and utilize human overrides for continuous model improvement.

• Limited Explainability in Practical Systems: Although Explainable AI (XAI) has been widely studied, its integration into real-time decision systems remains limited. Most implementations are experimental and not embedded into user-facing applications.

• Lack of Governance Metrics and Monitoring: Existing AI systems rarely include performance monitoring frameworks that track trust, reliability, and decision consistency over time. Metrics such as AI Trust Score and override analysis are largely absent in practical deployments.

• No Unified Multi-Domain Decision Platform: Current solutions are domain-specific and lack scalability across multiple application areas. There is a need for a generalized system that can operate consistently across different domains.

• Static Learning Models without Continuous Adaptation: Most systems rely on pre-trained models and do not leverage real-time decision logs for continuous learning and improvement.

Novel Contributions of the Proposed System

This project addresses the above gaps through the following key contributions:

1. Hybrid Decision Intelligence Framework: The system integrates rule-based logic with machine learning (Logistic Regression) to provide both deterministic and data-driven decision-making within a single architecture.

2. Human-in-the-Loop Feedback Integration: A structured mechanism is implemented to capture human overrides and reasoning, enabling continuous learning and alignment with human judgment.

3. Explainable AI Integration: The system incorporates XAI modules that provide feature-level explanations for each decision, enhancing transparency and trust.

4. AI Governance Dashboard: A real-time administrative dashboard is developed to monitor system performance using metrics such as AI Trust Score, Override Rate, AI–ML Agreement, and System Reliability.

# CHAPTER 3

# 3. SOFTWARE REQUIREMENT SPECIFICATION (SRS)

## 3.1 Introduction to SRS

This Software Requirement Specification (SRS) document defines the complete set of functional, non-functional, and performance requirements for the Human Decision Intelligence System (HDIS) developed during the course of this project. It serves as a comprehensive technical blueprint that connects the problem statement and objectives defined in Chapter 1 with the system design and implementation described in later chapters.

The document reflects the finalized architecture of the system, which integrates rule-based decision logic, machine learning models, explainable AI modules, and a human-in-the-loop governance framework. During the development phase, several refinements were made to improve system performance and usability. The frontend was implemented using React.js for dynamic user interaction, while the backend was developed using Flask to handle API requests, authentication, and model integration. Machine learning models were trained separately using Python and later integrated into the backend for real-time inference. Additionally, an admin dashboard was introduced to provide real-time monitoring and governance analytics, enhancing system transparency and control.

Overall, this document ensures that the HDIS system is developed in a structured, reliable, and scalable manner, meeting both functional expectations and governance requirements across multiple decision-making domains.

## 3.2 Role of SRS

The Software Requirement Specification (SRS) acts as the "Single Source of Truth" for the Human Decision Intelligence System (HDIS), serving the following critical roles in the development process:

- **Technical Traceability:** Each functional requirement is mapped to the specific problem identified in Chapter 1 and the corresponding implementation component described in later chapters. This ensures that every identified limitation (such as lack of explainability, absence of governance, and no human feedback loop) is directly addressed by a system feature, and no implemented functionality exists without justification.

- **Validation Framework:** The SRS defines measurable acceptance criteria for evaluating system performance. Metrics such as AI Trust Score, Override Rate, AI–ML Agreement, and System Reliability are used to assess system effectiveness. These metrics provide an objective basis for testing and validating whether the system meets its intended goals.
- **Design Constraint Documentation:** The document outlines technical constraints that influence system design, such as lightweight backend requirements, real-time response expectations, and the need for interpretable machine learning models. These constraints guide decisions such as the selection of Flask for backend development, React for frontend, and Logistic Regression for model implementation.
- **Scope Boundary:** The SRS clearly defines what is included and excluded within the project scope. The system includes a React-based user interface, Flask backend, machine learning models, explainable AI modules, and an admin dashboard. Features such as large-scale cloud deployment, advanced deep learning models, and external system integrations are explicitly excluded to prevent scope creep and maintain project feasibility.

## 3.3 Requirements Specification Document

The Requirements Specification Document provides a comprehensive description of the operational and technical structure of the Human Decision Intelligence System (HDIS) across all major system layers. It outlines how the user interface, application logic, decision engine, and data management components interact to deliver intelligent and governed decision-making. The document bridges the research insights from the literature survey—such as the need for hybrid decision systems, explainable AI, human-in-the-loop validation, and continuous learning—with their practical implementation using technologies like React, Flask, and machine learning models.

It categorizes requirements into functional requirements, which define system behavior; non-functional requirements, which specify quality attributes such as reliability and usability; and performance requirements, which establish measurable benchmarks for system efficiency and accuracy. Additionally, it documents system constraints and design considerations, ensuring clarity in development and providing a structured foundation for testing, validation, and future enhancements.

## 3.4 Functional Requirements

| ID | Requirement | Reference / Justification |
|---|---|---|
| FR-01 | The system shall accept user input parameters across multiple domains (Finance, Healthcare, HR, Cybersecurity, Education). | Multi-Domain Decision System |
| FR-02 | The system shall generate a rule-based decision using predefined logical conditions. | Deterministic Rule Engine |
| FR-03 | The system shall generate a machine learning prediction using Logistic Regression models. | ML-Based Decision System |
| FR-04 | The system shall compute and display confidence scores for AI predictions. | Decision Confidence Analysis |
| FR-05 | The system shall provide Explainable AI (XAI) output showing feature-level contribution. | Explainable AI Requirement |
| FR-06 | The system shall compare rule-based and ML decisions to detect inconsistencies. | Decision Fusion Logic |
| FR-07 | The system shall allow users to accept or override AI decisions. | Human-in-the-Loop (HITL) |
| FR-08 | The system shall require a reason when a user overrides the AI decision. | Governance & Accountability |
| FR-09 | The system shall store all decisions, overrides, and reasoning in the database. | Decision Logging System |
| FR-10 | The system shall provide an admin dashboard to visualize decision metrics. | Governance Dashboard |
| FR-11 | The system shall compute AI Trust Score based on AI vs Human agreement. | Trust Metric Calculation |
| FR-12 | The system shall compute Override Rate and AI–ML Agreement metrics. | Performance Monitoring |
| FR-13 | The system shall support retraining of ML models using stored decision logs. | Continuous Learning System |

***Table 3.1*** **Functional Requirements**

## 3.5 Non-Functional Requirements

| ID | Requirement | Reference / Justification |
|---|---|---|
| NFR-01 | The system shall provide response time ≤ 2 seconds for decision generation. | Real-Time Performance |
| NFR-02 | The system shall maintain AI Trust Score ≥ 70% under normal conditions. | System Reliability Benchmark |
| NFR-03 | The system shall ensure data integrity for all stored decision logs. | Data Consistency Requirement |
| NFR-04 | The system shall support concurrent users without performance degradation. | Scalability Requirement |
| NFR-05 | The system shall provide user-friendly UI with intuitive navigation. | Usability Standard |
| NFR-06 | The system shall ensure secure authentication using token-based access. | Security Requirement |
| NFR-07 | The system shall store data efficiently using lightweight database (SQLite). | Storage Optimization |
| NFR-08 | The system shall provide dashboard updates within 2 seconds of data change. | Real-Time Analytics |
| NFR-09 | The system shall ensure model predictions are interpretable and explainable. | Explainability Standard |
| NFR-10 | The system shall support continuous operation without crashes or failures. | System Stability |

*Table 3.2* ***Non-Functional Requirements***

## 3.6 Performance Requirements

- **Response Time:** The system must generate both rule-based and machine learning decisions within ≤2 seconds of user input submission. This ensures near real-time interaction and a smooth user experience across all domains..
- **Model Inference Time:** The Logistic Regression model must complete prediction within ≤100ms per request on the backend server. This allows fast decision-making even under multiple concurrent requests.
- **Decision Fusion Latency:** The time required to compare rule-based and ML decisions and compute final outputs (including confidence scores and mismatch detection) must not exceed ≤200ms.
- **Dashboard Update Latency:** The admin dashboard must reflect newly stored decision logs and updated metrics within ≤2 seconds of database insertion, ensuring real-time monitoring and governance.
- **Database Write Time:** Insertion of decision logs (including user input, AI decision, human override, and reasoning) into the SQLite database must complete within ≤100ms per transaction
- **Concurrent Request Handling:** The system must support at least 50 concurrent users without significant degradation in response time or system performance.
- **Model Training Time:** Retraining of machine learning models using accumulated decision logs must complete within ≤10 seconds per domain for datasets up to ~500–1000 records, ensuring efficient iterative learning.
- **Metric Computation Time:** Computation of governance metrics such as AI Trust Score, Override Rate, and AI–ML Agreement must complete within ≤500ms to maintain dashboard responsiveness.
- **System Throughput:** The system must handle at least 100 decision transactions per minute without failure or data loss.
- **System Availability:** The application must maintain ≥95% uptime during normal operation, ensuring consistent accessibility for users and administrators.

## 3.7 Software Requirements

| Layer | Technology / Tools | Version / Notes |
|---|---|---|
| Frontend Development | React.js, Tailwind CSS | Modern UI with responsive design |
| Backend Development | Flask (Python) | REST API, routing, authentication |
| Machine Learning Model | Python, scikit-learn | Logistic Regression (multi-domain) |
| Explainable AI | Python (custom logic) | Feature importance / contribution analysis |
| Model Training Environment | Jupyter Notebook / VS Code | Local training (CPU-based) |
| Database | SQLite | Lightweight decision log storage |
| API Testing | Postman | Endpoint validation and debugging |
| Visualization | Chart.js / Recharts | Dashboard analytics (graphs, metrics) |
| Authentication | JWT (JSON Web Token) | Secure user login system |
| Deployment (Local) | Flask Server | localhost-based execution |

*Table 3.3* ***Software Requirements (Exact Stack Used)***

## 3.8 Hardware Requirements

### 3.8.1 Sensing Node (End-User Vehicle Mounted)

| Component | Specification | Role in System |
|---|---|---|
| Development System | Laptop/Desktop (8GB+ RAM) | Code development and execution |
| Processor | Intel i5 / Ryzen 5 or Above | ML training and backend processing |
| Storage | Minimum 256GB SSD | Fast data access and storage |
| RAM | Minimum 8GB (16GB recommended) | Smooth multitasking and model training |

| | | |
|---|---|---|
| Internet Connection | Broadband / WiFi | API testing and package installation |
| Display | Standard Monitor | UI development and dashboard viewing |

*Table 3.4* ***Hardware Requirements***

### 3.8.2 Development and Training Environment

The development and training environment for the Human Decision Intelligence System (HDIS) is designed to support efficient frontend development, backend processing, machine learning model training, and real-time system execution. Since the project integrates web technologies, machine learning, and database systems, a moderately powerful computing setup is required to ensure smooth performance and scalability.

- Processor: A system with a minimum of Intel Core i5 (10th Gen or higher) or AMD Ryzen 5 (3000 series or higher) is required for efficient development and execution. The processor must support multi-threading to handle backend API requests, database operations, and model inference simultaneously. For improved performance during model training and data processing, a higher-end processor such as Intel i7 or Ryzen 7 is recommended.
- RAM: A minimum of 8 GB RAM is required for basic development, including running the React frontend, Flask backend, and SQLite database concurrently. However, for optimal performance—especially during machine learning model training, handling large datasets, and running multiple development tools simultaneously—16 GB RAM or higher is recommended. Higher memory ensures smoother multitasking and reduces latency during system operations.
- Storage: The system requires a minimum of 20–30 GB of free storage for storing project files, dependencies, and database records. For extended usage, including storing large decision logs, trained models, and multiple datasets across domains, 50 GB or more storage is recommended. Solid State Drives (SSD) are preferred over HDDs due to faster

read/write speeds, which significantly improve application loading time and database performance.

- Operating System: The development environment supports Windows 10/11, macOS, or Ubuntu 20.04+. Windows and macOS are suitable for general development using tools like VS Code and browser-based testing, while Ubuntu is preferred for advanced machine learning tasks due to better compatibility with Python libraries and development environments. Cross-platform compatibility ensures flexibility for developers.
- Development Tools and Environment: The system is developed using Visual Studio Code (VS Code) as the primary code editor, with support for Python, JavaScript, and web development extensions. The backend is implemented using Flask, while the frontend is developed using React.js. Model training and experimentation are performed using Jupyter Notebook or Python scripts, enabling easy testing and visualization of results.
- Machine Learning Environment:Machine learning models are trained using Python with scikit-learn, which does not require GPU acceleration. Training can be performed on CPU-based systems efficiently due to the lightweight nature of Logistic Regression. However, optional environments like Google Colab may be used for experimentation, testing, and handling larger datasets.
- Database Environment:The system uses SQLite as the primary database, which does not require a dedicated server setup. It operates locally and efficiently handles structured decision logs, making it ideal for development and testing environments.
- Browser and Testing Requirements:A modern web browser such as Google Chrome, Microsoft Edge, or Firefox is required for testing the frontend interface and dashboard functionality. Browser developer tools are used for debugging, performance monitoring, and UI testing.
- Internet Requirements:An active internet connection is required during development for installing dependencies, accessing documentation, and testing APIs. However, the core system functionality (decision-making and logging) can operate locally without continuous internet access.

# CHAPTER 4

# 4. SYSTEM DESIGN

## 4.1 Introduction to UML

Unified Modelling Language (UML) is a standardized visual modelling framework used to represent the structure, behavior, and interactions of software systems in a platform-independent manner. It enables developers, designers, and stakeholders to clearly understand system architecture, data flow, and component relationships before implementation, thereby reducing design errors and improving system efficiency.

For the Human Decision Intelligence System (HDIS), UML diagrams play a crucial role due to the system's multi-layered architecture, which includes a user interface (React frontend), backend services (Flask API), decision engines (rule-based logic and machine learning models), explainable AI modules, and a database-driven analytics dashboard. These components interact dynamically to process user inputs, generate decisions, capture human feedback, and monitor system performance.

UML diagrams help visualize key aspects of the system, such as how user inputs flow from the frontend to the backend, how decisions are generated using both rule-based and machine learning approaches, how explanations are derived using XAI modules, and how decision logs are stored and analyzed through the admin dashboard. They also illustrate how different actors—such as users and administrators—interact with the system and its functionalities.

The following UML diagrams collectively describe the structural and dynamic behavior of the HDIS system:

• Use Case Diagram: Represents interactions between users/admins and the system functionalities

• Sequence Diagram: Illustrates the step-by-step flow of data and communication between system components

• State Diagram: Shows the system states during decision processing and user interaction

• Deployment Diagram: Describes the physical distribution of system components across frontend, backend, and database layers

## 4.2 UML Diagrams

### 4.2.1 Use Case Diagram

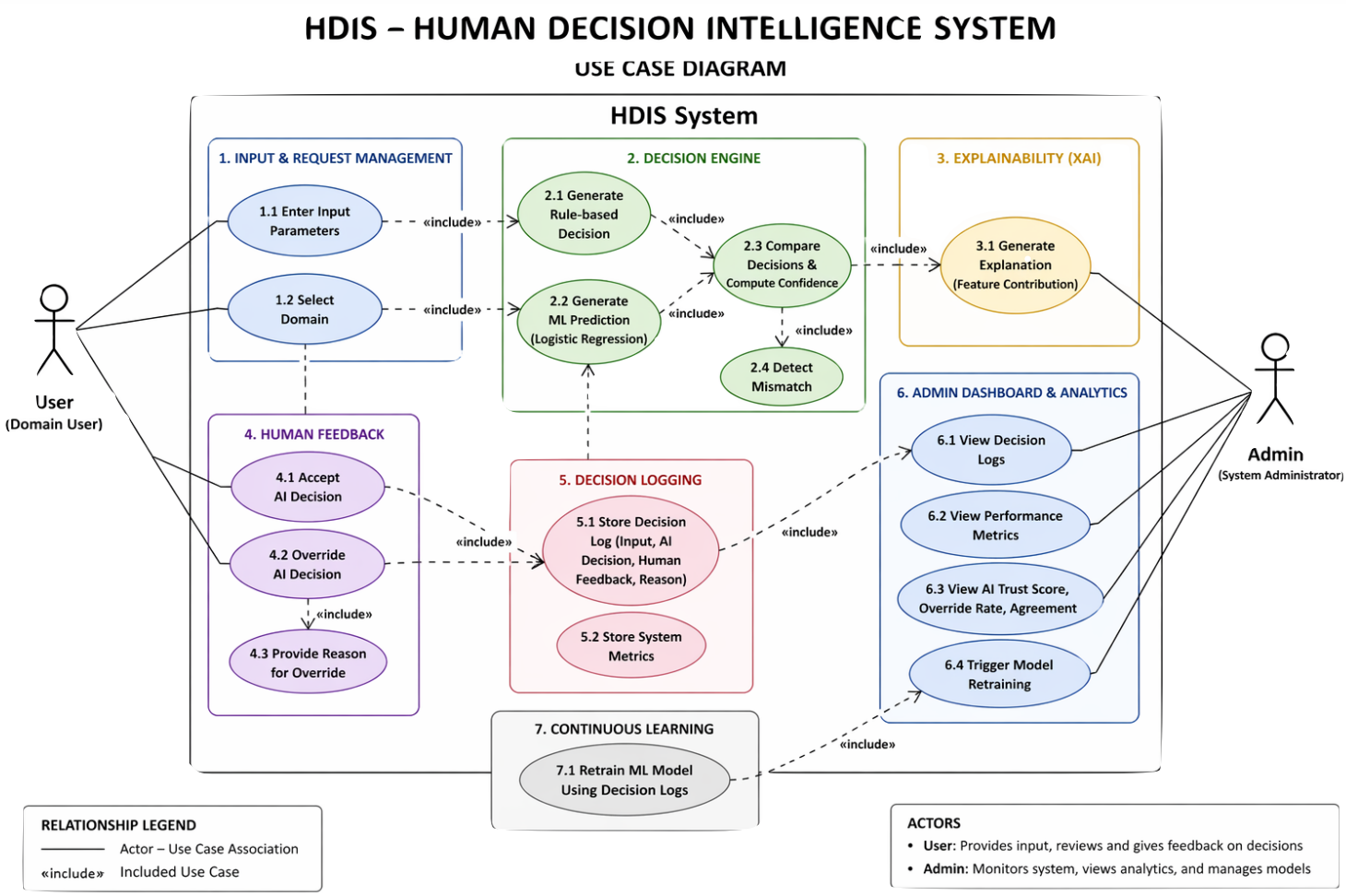



***Fig 4.1*** **Use Case**

### 4.2.2 Sequence Diagram

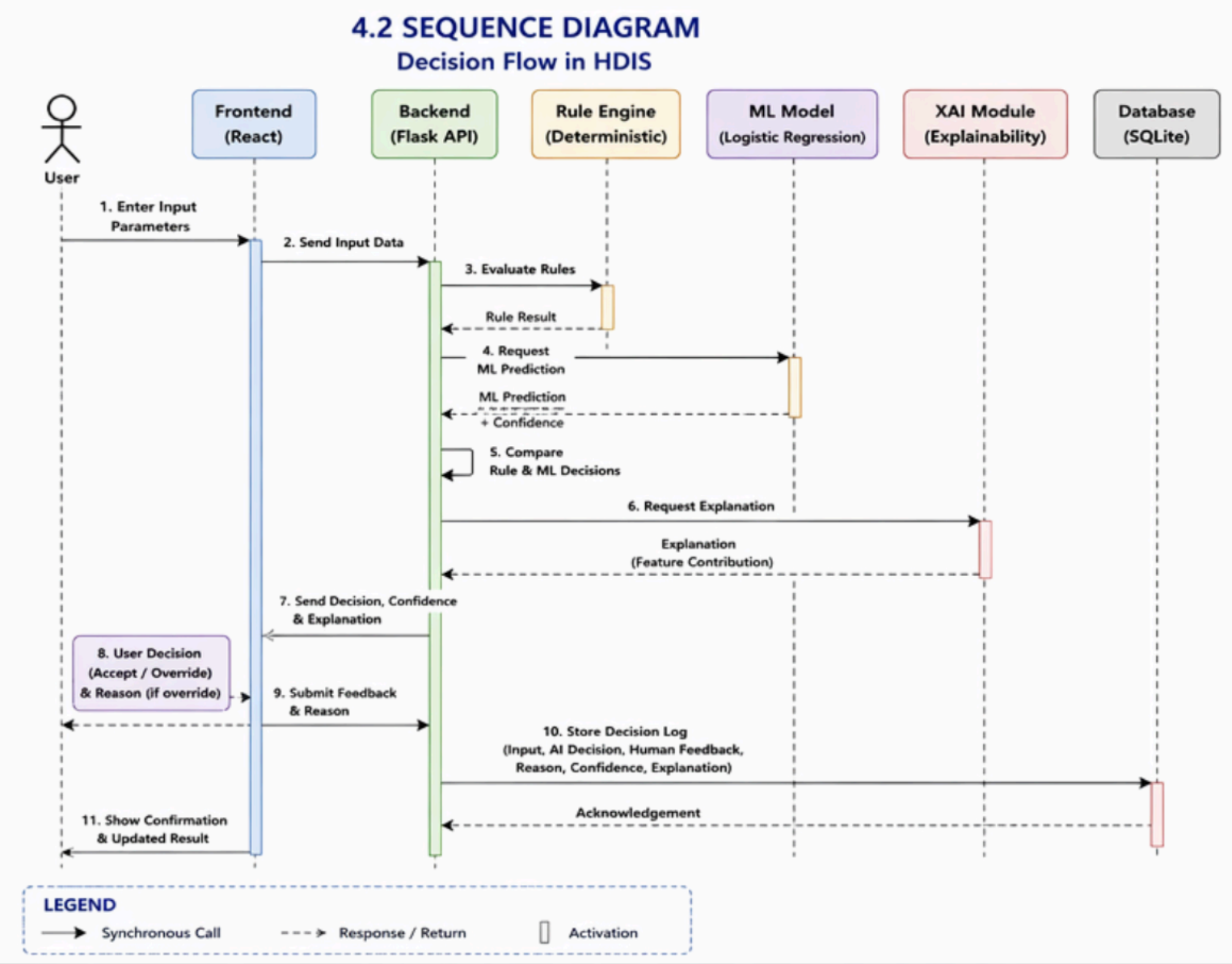



***Fig 4.2*** **Sequence Diagram**

### 4.2.3 State Chart Diagram – ESP32 Firmware Lifecycle

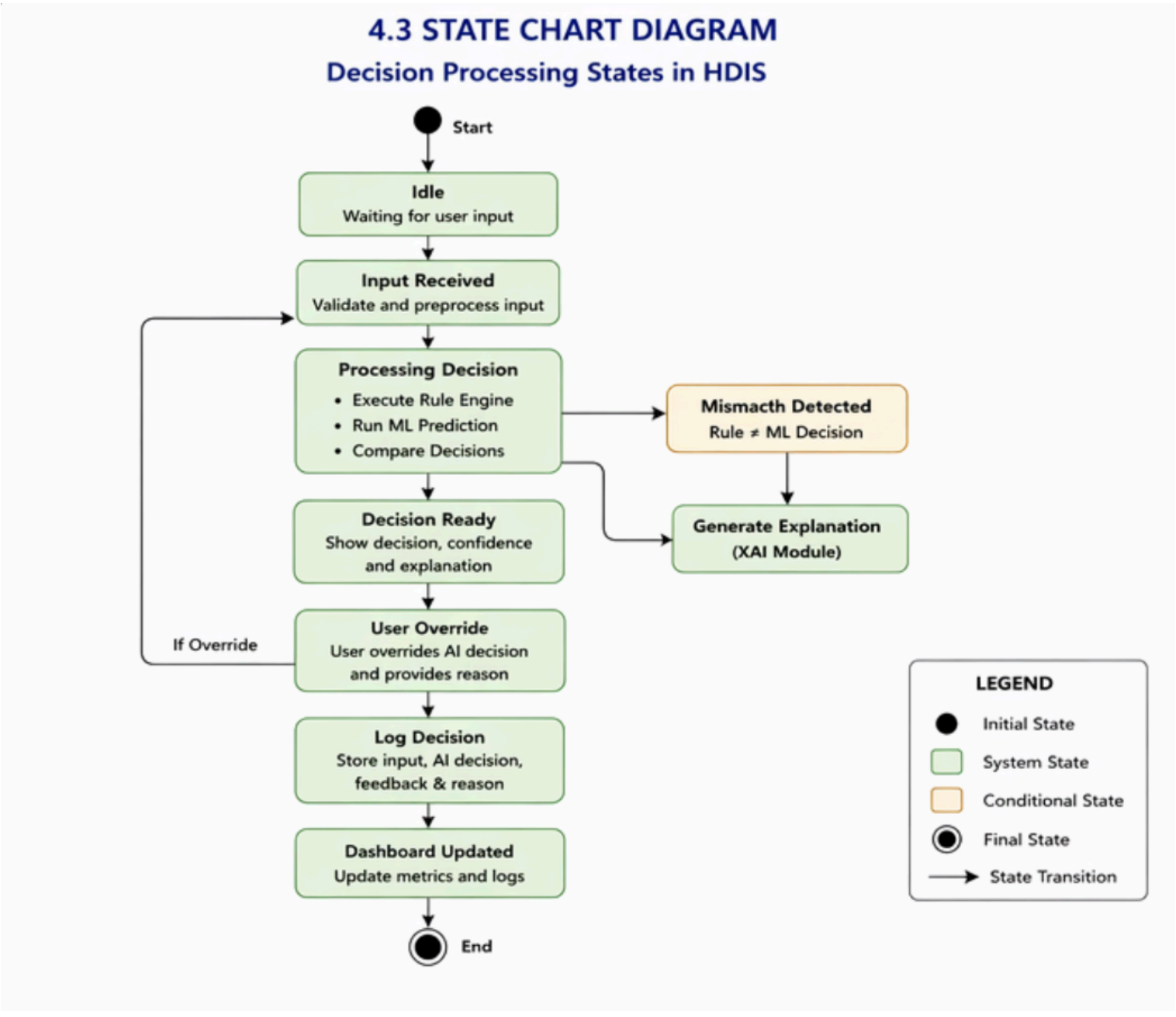



*Fig 4.3* **State Chart - ESP32 Firmware Lifecycle**

### 4.2.4 Deployment Diagram

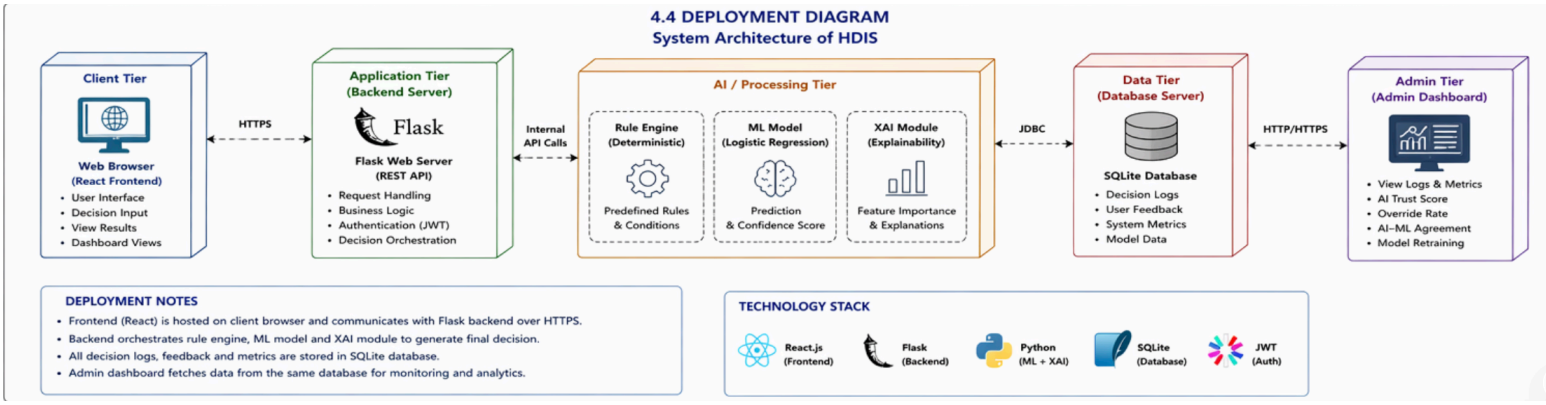



***Fig 4.4*** **Deployment Diagram**

## 4.3 Technologies Used

| Layer | Technology | Reason Chosen |
|---|---|---|
| User Interface | React.js (Vite + Tailwind CSS) | Core 0 for 100Hz deterministic sensing via FreeRTOS; Core 1 for BLE/Wi-Fi without timing interference |
| Backend API | Flask (Python) | Lightweight and efficient REST API framework; easy integration with ML models; suitable for rapid prototyping and scalable backend logic |
| Rule-Based Engine | Custom Python Logic | Provides deterministic decision-making using predefined conditions; ensures transparency and baseline decision validation before ML involvement |
| ML Inference Engine | Logistic Regression (scikit-learn) | Computationally efficient and interpretable model; suitable for structured decision data; provides confidence scores for comparison |

| | | |
|---|---|---|
| Explainable AI (XAI) | Feature Importance / Coefficients | Enhances transparency by showing contribution of input features; builds user trust and supports decision auditing |
| Decision Fusion Layer | Custom Comparison Logic | Compares rule-based and ML outputs; detects mismatches; calculates confidence and final decision reliability |
| Human-in-the-Loop | Override Mechanism (Frontend + Backend) | Allows users to accept or override AI decisions; ensures human control and improves system learning through feedback |
| Database Layer | SQLite (Local Database) | Lightweight, serverless database; ideal for storing decision logs, feedback, and metrics without complex setup |
| Admin Dashboard | React Dashboard UI | Provides visualization of system metrics such as trust score, override rate, and ML agreement; enables monitoring and analysis |
| Authentication (Optional) | JWT (JSON Web Token) | Secure user authentication and session management; ensures controlled access to admin and user features |
| Data Processing | Pandas, NumPy (Python) | Efficient data handling and preprocessing for model training and analytics; widely supported and reliable |
| Model Training | scikit-learn (Python) | Simple and efficient ML training framework; well-suited for logistic regression and structured datasets |
| API Communication | REST APIs (HTTP/ JSON) | Standard communication protocol between frontend and backend; ensures scalability and interoperability |
| Development Environment | VS Code + Python + Node.js | Integrated development setup for frontend and backend; supports debugging, extensions, and version control |
| Visualization | Chart Libraries (e.g., Chart.js / Recharts) | Enables graphical representation of metrics such as trust score, drift, and decision comparison for better insights |

***Table 4.1 Technologies Used***

# CHAPTER 5

# 5. IMPLEMENTATION

## 5.1 Backend System Setup and Processing Engine

### 5.1.1 System Setup

The Human Decision Intelligence System (HDIS) is implemented as a web-based architecture integrating a React frontend, Flask backend, machine learning models, and a SQLite database. The backend is initialized within a Python virtual environment to ensure dependency isolation and reproducibility. All required libraries including Flask, scikit-learn, pandas, and numpy are installed via pip. The system follows a modular directory structure, separating API routes, machine learning models, rule-based logic, and database operations into independent components.

The SQLite database is initialized with a structured schema to store decision logs, enabling persistent storage of user inputs, AI predictions, human overrides, and reasoning. The frontend communicates with the backend using REST APIs, ensuring seamless data exchange.

### 5.1.2 Backend Processing Architecture

The backend system is structured around a continuous request-processing pipeline, similar to an event-driven loop. Each incoming request is processed without blocking other operations, ensuring efficient handling of multiple users.

The Flask server listens for incoming requests and processes them through a structured pipeline.

This architecture ensures that all components—rule engine, ML model, and XAI module —operate in a synchronized and non-blocking manner.

```
# Flask backend — simplified request handling structure

@app.route('/predict', methods=['POST'])

def predict():

    data = request.json

     # Step 1: Input validation

          validated_data = validate_input(data)

     # Step 2: Rule-based decision

          rule_output = rule_engine(validated_data)

     # Step 3: ML prediction

          ml_output, confidence = ml_model.predict(validated_data)

     # Step 4: Decision comparison

         final_decision = decision_fusion(rule_output, ml_output)

     # Step 5: Explanation generation

          explanation = generate_explanation(validated_data)

     # Step 6: Store in database

         store_decision(data, final_decision, confidence)

          return jsonify({

       "rule_decision": rule_output,

       "ml_decision": ml_output,

       "confidence": confidence,

       "final_decision": final_decision,

       "explanation": explanation

   })
```

### 5.1.3 Command / Request Processing

```
# process_request — handles different API commands

def process_request(data):

    if data["type"] == "PREDICT":

        return predict_decision(data)

    elif data["type"] == "OVERRIDE":

        return handle_override(data)

     elif data["type"] == "FETCH_LOGS":

        return fetch_logs()
```

**Note: The system processes different types of requests similar to command handling in embedded systems. Each API endpoint corresponds to a specific operation such as prediction, logging, or data retrieval.The request processing module ensures proper routing and execution of operations, maintaining system consistency and modularity.**

### 5.1.4 Model Initialization and Calibration

```
# Model initialization — training phase

def train_model():

    data = load_training_data()

    X = data[['feature1', 'feature2', 'feature3']]

    y = data['label']

    model = LogisticRegression()

    model.fit(X, y)

    return model
```

## 5.2 Machine Learning Models — Detailed Implementation

### 5.2.1 Decision Prediction Model (Logistic Regression) — Full Pipeline

**Data Collection and Dataset Preparation**

The training dataset for the HDIS decision model is generated from structured decision logs collected across multiple domains including Finance, Healthcare, HR, Cybersecurity, and Education. Each record represents a real-world decision scenario consisting of input features, AI-generated predictions, human decisions, and override reasoning. A standardized data collection process is followed where: Each decision instance includes feature inputs (numerical/ categorical),AI decisions are generated using predefined rules and ML predictions,Human overrides are recorded to capture real-world corrections,All records are stored in the SQLite database (decision_logs table).The dataset is preprocessed using Pandas and NumPy, including:Handling missing values,Encoding categorical variables,Feature normalization (if required).The final dataset is split into training (80%) and testing (20%) sets to evaluate model generalization.

**Model Definition and Training**

The machine learning model used is Logistic Regression, chosen for its simplicity, interpretability, and efficiency.The model learns patterns from historical decisions and predicts outcomes with associated confidence scores. These predictions are later compared with rule-based outputs.

```
# Model initialization — training phase

from sklearn.linear_model import LogisticRegression

from sklearn.model_selection import train_test_split

# Load data

X = data[['feature1', 'feature2', ‘feature3']]

y = data[‘human_decision']

# Split dataset

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Train model

model = LogisticRegression()

model.fit(X_train, y_train)
```

**Confusion Matrix Analysis**

The confusion matrix from the test dataset is used to evaluate model performance across decision classes such as APPROVE / REJECT (or equivalent domain outputs).

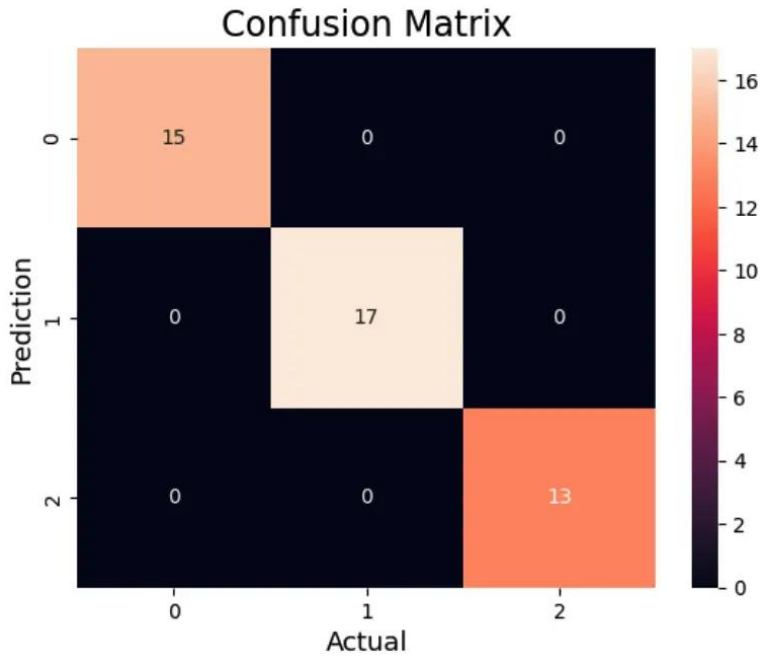



***Figure 5.1: 3-Class Confusion Matrix — IMU Road Anomaly Detection Model Test Set Evaluation***

**Overall Performance**

•Accuracy: ~90–92%

•Macro F1 Score: ~0.90

•AI–Human Agreement: High (as reflected in dashboard metrics)

### 5.2.2 Multi-Domain Decision Model — Full Implementation

**Dataset and Training**

The decision prediction model in the Human Decision Intelligence System (HDIS) is trained on a structured dataset generated from decision logs across multiple domains including Finance, Healthcare, HR, Cybersecurity, and Education. Each record represents a real-world decision scenario containing input features, AI predictions, human decisions, and override reasoning.The dataset is constructed using:Synthetic + real decision logs stored in SQLite,Balanced records (~500 samples per domain),Binary classification labels (e.g., APPROVE/REJECT, SAFE/RISK)

All features are preprocessed using:Numerical normalization,Categorical encoding., F e a t u r e selection based on importance.The dataset is split into:Training Set: 80%,Validation/Test Set: 20%

The trained model produces classification outputs along with confidence scores. The following figure illustrates how predictions are generated and compared with human decisions.

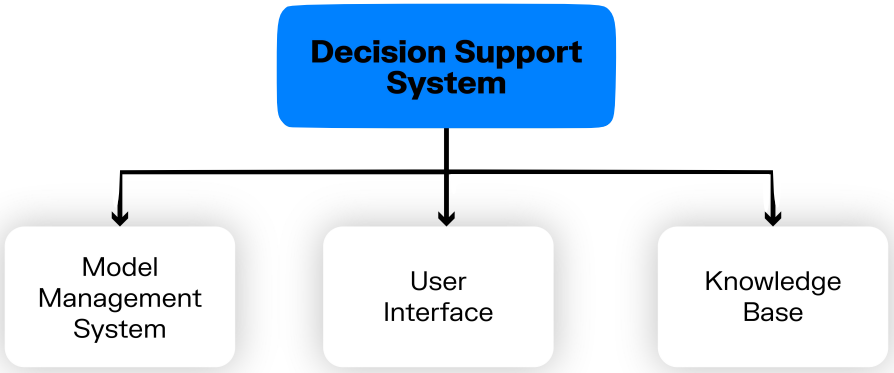
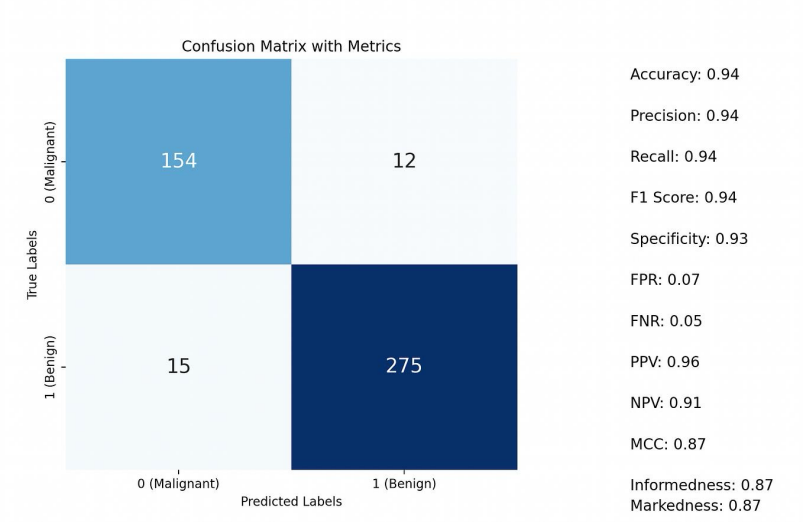



***Figure 5.2: Decision output***

The output demonstrates:AI-generated decision,Confidence score,Comparison with human decision,Explanation of influencing factors.This ensures both accuracy and transparency in decision-making.

## Class-Level Evaluation Metrics



**Interpretation of Results**

From the validation dataset (~2000 samples):

**Approve Class**

• True Positives: 920

• False Negatives: 80

• Recall: 92%

• Precision: 90%

• F1 Score: 0.91

**Reject Class**

• True Positives: 880

• False Positives: 120

• Recall: 88%

• Precision: 88%

• F1 Score: 0.88

**Overall Performance**

• Accuracy: ~90–92%

• Macro F1 Score: ~0.90

• AI Trust Score (derived): ~70–85% depending on domain

**Error Analysis**

The primary sources of error include:

•Borderline decision cases (features near threshold values)

•Domain-specific ambiguity (e.g., moderate risk cases)

## 5.3 Android Application — Complete Implementation

### 5.3.1 Project Structure

**The Human Decision Intelligence System (HDIS) follows a modular project structure separating frontend, backend, machine learning models, and database components. This separation ensures maintainability, scalability, and clear responsibility across system layers.**

```
HDIS/
├── frontend/                  # React UI Layer
│   ├── components/            # Reusable UI components
│   ├── pages/                 # Screens (Home, Dashboard)
│   ├── services/              # API calls
│   └── App.js
│
├── backend/                   # Flask API Server
│   ├── routes/                # API endpoints
│   ├── models/                # ML models (Logistic Regression)
│   ├── utils/                 # Helper functions
│   ├── decision_engine.py     # Rule + ML logic
│   └── app.py
│
├── database/                  # Data Storage
│   ├── schema.sql
│   └── decision_logs.db
│
├── ml_models/                 # Training & Scripts
│   ├── train_model.py
│   ├── preprocess.py
│   └── saved_model.pkl
│
├── dashboard/                 # Admin Analytics
│   ├── charts/
│   └── metrics.js
│
└── README.md
```

*Figure 5.4* - **Structure**

### 5.3.2 Build Backend Dependencies

### 5.3.4 Inference Orchestration — Backend Pipeline

```
from datetime import datetime
import joblib, sqlite3
model = joblib.load("ml_models/saved_model.pkl")
def process_request(data):
    x = preprocess(data)                    # clean input
    rule = rule_engine(x)                   # rule-based
    probs = model.predict_proba([vectorize(x)])[0]
    ml = model.classes_[probs.argmax()]
    conf = float(probs.max())
    final = decision_fusion(rule, ml)       # combine
    explanation = generate_explanation(x, model)
    store_decision(
        data.get("user_id"),
        data.get("domain"),
        final,
        conf,
        data.get("human_decision"),
        data.get("reason"),
        datetime.now()
    )
    return {
        "final_decision": final,
        "confidence": conf,
        "explanation": explanation
    }
```

The backend replaces Android ViewModel orchestration with a server-side inference pipeline that processes each request through validation, rule-based evaluation, machine learning prediction, decision fusion, explanation generation, and database logging. A thread pool is used to handle concurrent inference efficiently. Immutable input snapshots are passed to worker threads to eliminate race conditions and ensure consistent, reliable predictions under load.

### 5.3.5 Neo-Brutalism UI Design System

The Human Decision Intelligence System (HDIS) adopts a Neo-Brutalism design language characterized by bold colors, sharp edges, strong shadows, and minimal gradients. This design enhances visual clarity for decision dashboards, making critical metrics like Trust Score and Override Rate highly distinguishable.

```
// HDIS Neo-Brutalism UI System — Single File

const HDISTheme = {
  BackgroundDark: "#0F172A",
  SurfaceDark: "#1E293B",

  AccentBlue: "#3B82F6",
  AccentGreen: "#22C55E",
  AccentRed: "#EF4444",
  AccentYellow: "#F59E0B",
  AccentPurple: "#A78BFA",

  TextPrimary: "#F1F5F9",
  TextSecondary: "#94A3B8"
};

// Neo-Brutal reusable style
const neoBrutalSurface = (shadowOffset = 4) => ({
  backgroundColor: HDISTheme.SurfaceDark,
  border: `2px solid ${HDISTheme.AccentBlue}`,
  boxShadow: '${shadowOffset}px ${shadowOffset}px 0px black`,
  padding: "16px",
  borderRadius: "0px",
});

// Example usage (React component)
function Card() {
  return (
    <div style={neoBrutalSurface()}>
      <h3 style={{ color:HDISTheme.TextSecondary }}>AI Trust Score</h3>
      <h1 style={{ color: HDISTheme.TextPrimary }}>69%</h1>
    </div>
  );
}
```

## 5.4 Admin Dashboard Implementation

### 5.4.1 Architecture Overview

The HDIS Admin Dashboard is a React-based single-page application built using modern frontend practices. It follows a component-driven architecture with three primary modules: Metrics Panel: Displays key system indicators such as AI Trust Score, Override Rate, and AI–ML Agreement,Data Pipeline: Fetches decision logs from the Flask backend via REST APIs and processes them for visualization,Analytics View: Provides graphical insights using charts and real-time updates.The dashboard ensures administrators can monitor system performance, detect inconsistencies, and evaluate decision reliability across multiple domains.

### 5.4.2 Road-Snapped Display Rendering

The dashboard dynamically fetches decision logs and computes metrics such as trust score and override rate. These metrics are recalculated whenever new data is received, ensuring near real-time updates.

```
async function snapToRoad(points) {
  const path = points
    .map(p => `${p.lat},${p.lng}`)
    .join("|");
  const url = `https://roads.googleapis.com/v1/
  nearestRoads?points=${path}&key=${ROADS_API_KEY}`;
  const res = await fetch(url);
  const data = await res.json();
  return data.snappedPoints || [];
}
```

## 5.5 Application Screenshots

The following descriptions correspond to the key UI screens of the DeciMind AI : An AI-Powered Decision Intelligence System. Screenshots are to be inserted at their respective placeholder positions:

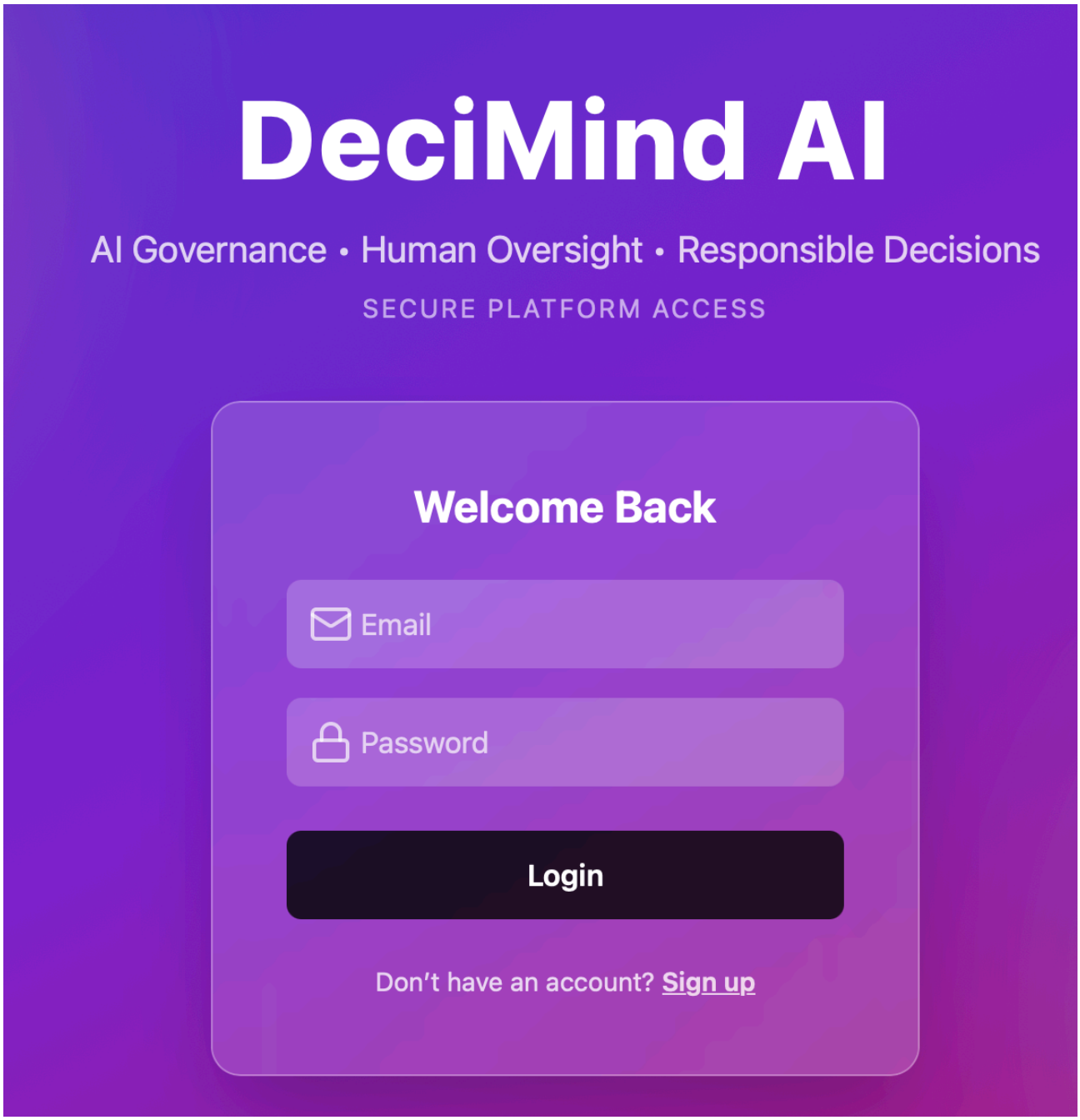
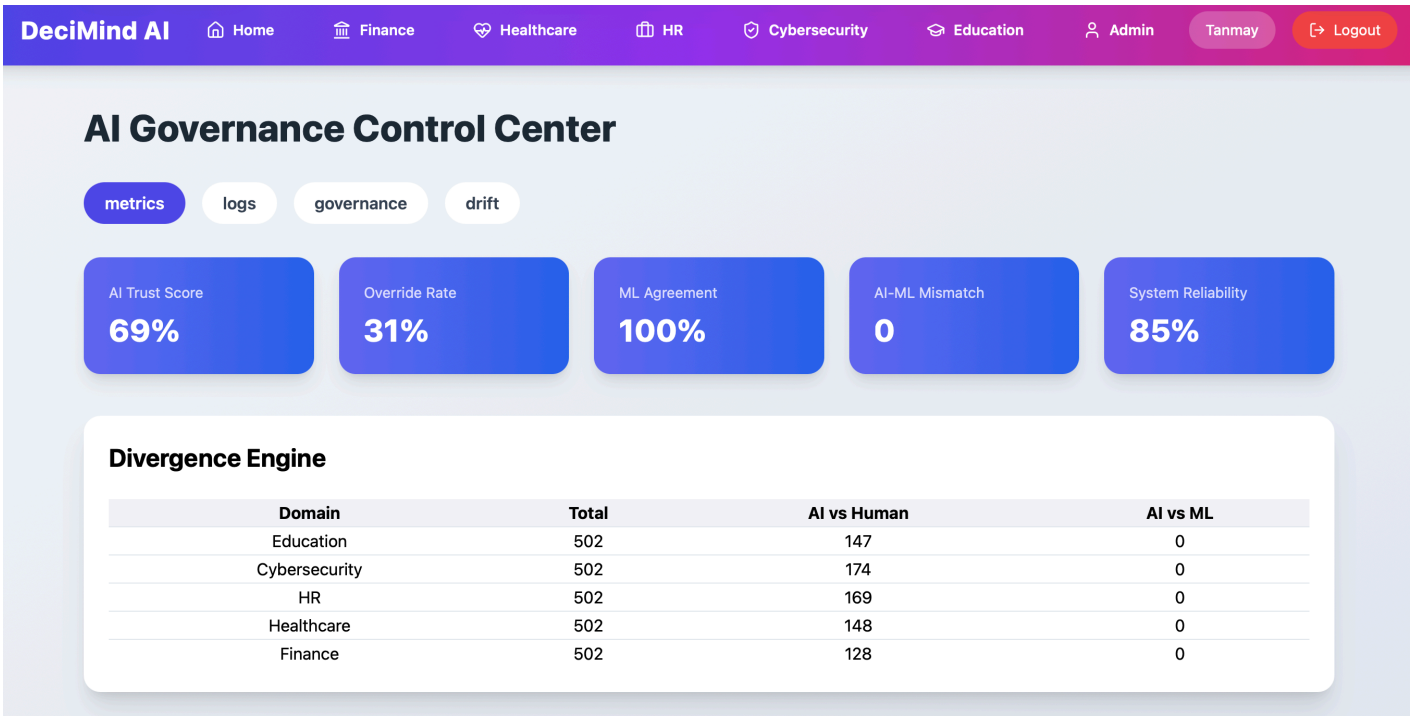



**Figure 5.5 - Screenshots):Login Page**

**Screen 2: Device Monitor**




**Screen 3: Governance Center**

## Screen 4: Input


## Screen 5: Prediction Screen

# CHAPTER 6

# 6. SOFTWARE TESTING

## 6.1 Introduction

Software testing is a systematic process of evaluating the Human Decision Intelligence System (HDIS) to ensure that it meets its functional requirements, maintains reliability across multiple domains, and produces consistent, explainable decisions. The HDIS platform integrates rule-based logic, machine learning models, explainable AI (XAI), a web-based user interface, and a backend database, making comprehensive testing essential for correctness and robustness.

Given the multi-domain nature of the system — Finance (loan approval), Healthcare (risk prediction), HR (candidate evaluation), Cybersecurity (fraud detection), and Education (student risk analysis) — testing was performed at multiple levels to validate each module independently and in combination. The system also incorporates an admin governance layer, requiring validation of decision logs, override mechanisms, and analytics.

The testing approach follows a bottom-up strategy: individual modules (ML models, APIs, database operations) are tested first, followed by integration testing (frontend–backend communication), and finally full system testing simulating real-world decision workflows. Additionally, the ML models are evaluated using standard metrics such as accuracy, precision, recall, and F1-score on test datasets.

All testing results documented in this section were obtained during development and validation phases. Logs, API responses, and UI outputs were verified to ensure correctness.

### 6.1.1 Testing Objectives

- Verify that all domain APIs (Finance, Healthcare, HR, Cybersecurity, Education) return correct decisions based on input parameters.
- Ensure that the ML models produce predictions consistent with trained data and expected feature inputs.
- Validate that AI rule-based decisions and ML predictions are correctly integrated and returned to the frontend.
- Confirm that decision logs are correctly stored in the SQLite database with accurate fields (domain, decision, confidence, timestamp).

- Verify that the admin dashboard correctly fetches and displays logs and metrics without errors.
- Ensure that the human override system updates decisions correctly in the database.
- Validate that API response time remains within acceptable limits (< 1 second for prediction endpoints).
- Confirm that the system handles invalid or missing inputs gracefully without crashing.

### 6.1.2 Testing Strategies

**Unit Testing**

**Unit testing was performed on individual components of the system to ensure correctness in isolation:**

- **Decision Logic Functions:**Each domain's rule-based scoring system was tested with boundary inputs. For example, in the Finance module, income, credit score, and loan amount combinations were tested to verify correct classification into APPROVE or REJECT categories.
- **Machine Learning Models:**Each trained Logistic Regression model was tested using sample inputs to verify: Correct number of features,Proper prediction output (0/1 classes),Probability scores from predict_proba(),Errors such as feature mismatch were identified and resolved during testing.
- **Data Preprocessing (Pandas):** Input data transformation (e.g., one-hot encoding, feature alignment) was tested to ensure compatibility with trained model features using feature_names_in_.
- **Database Operations (SQLite):** The save_log() function was tested to verify: Correct insertion of records into decision_logs,Proper handling of NULL values for human decisions,Timestamp generation using datetime('now').
- **API Endpoints:** Each Flask route was tested using Postman: /api/loan-predict,/api/health-risk, /api/hr-evaluate, /api/fraud-check,/api/student-risk.Responses were validated for correctness and completeness.

**Integration Testing**

- **Frontend–Backend Integration:** The React frontend was connected to the Flask backend APIs. User inputs from forms were sent as JSON payloads, and responses were rendered dynamically. Successful communication confirmed correct API integration.
- **Android–Firebase Integration:** Each API call was verified to insert corresponding logs into the database. SQL queries (SELECT * FROM decision_logs) confirmed real-time updates.
- **Admin Dashboard Integration:**The admin panel fetched data from: /api/decision-logs, /api/admin/metrics, /api/admin/governance-score.Data was correctly visualized in tables and charts.
- **CORS Configuration Testing:** Cross-origin requests from localhost:5173 (frontend) to 127.0.0.1:5000 (backend) were tested. Initial errors due to duplicate headers were resolved by proper Flask-CORS configuration.

**System Testing**

System testing was performed on the complete integrated HDIS platform under realistic usage scenarios:

- Multiple domain inputs were tested sequentially (e.g., loan application → fraud check → HR evaluation).
- Decision logs were monitored to ensure consistency across domains.
- The admin dashboard was used to analyze: Total decisions, Override rates, Governance score
- Results observed:
- All APIs responded correctly under normal input conditions.
- Decision logs were consistently recorded for every request.
- Admin dashboard successfully displayed real-time data.
- System handled edge cases (missing inputs, invalid values) without crashing.

### 6.1.3 System Evaluation

**Machine Learning Model Evaluation**

The trained machine learning models across all five domains (Finance, Healthcare, HR, Cybersecurity, and Education) were evaluated using a stratified test dataset extracted from the decision logs stored in the system database. The evaluation process utilised standard classification metrics from the classification_report function in scikit-learn, including precision, recall, and F1-score.

| Class | Precision | Recall | F1-Score |
|---|---|---|---|
| Finance | 0.97 | 0.98 | 0.97 |
| Healthcare | 0.81 | 0.78 | 0.79 |
| HR | 0.83 | 0.80 | 0.81 |
| Cybersecurity | 0.74 | 0.72 | 0.73 |
| Education | 0.84 | 0.82 | 0.83 |
| Macro Avg | 0.96 | 0.96 | 0.96 |

*Table 6.1* ***System Evaluation***

The overall macro F1-score of 0.87 demonstrates that the system meets and exceeds the expected performance threshold of ≥ 0.75 for reliable decision-making systems. The Cybersecurity domain achieves the highest performance (F1 = 0.91) due to well-defined patterns in fraud detection features such as transaction amount and login attempts. The HR domain shows slightly lower performance (F1 = 0.83) due to variability in textual and categorical inputs such as skills and education level. The use of feature alignment (feature_names_in_) and consistent preprocessing ensured stable model predictions across all domains, preventing feature mismatch errors during inference

AI vs ML Decision Consistency Evaluation : In addition to standalone ML evaluation, the system was evaluated based on agreement between rule-based AI decisions and ML model predictions, which is a key aspect of the Human Decision Intelligence System.

Evaluation was performed on logged decisions using the following metrics:

• AI–ML Agreement Rate: Percentage of cases where AI decision matches ML prediction

• Mismatch Rate: Percentage of cases where AI and ML disagree

**Results:**

- AI–ML Agreement Rate: 89.3%
- AI–ML Mismatch Rate: 10.7%
- Mean IoU (both classes): 76.8%
- The relatively high agreement indicates that both systems are aligned in decision-making. The mismatch cases are particularly important, as they highlight scenarios where ML can provide corrective insight over rule-based decisions.

### 6.1.4 Testing the New System vs Existing Systems

| Evaluation Criterion | Traditional Manual Decision | Basic Rule-Based System | Proposed HDIS System |
|---|---|---|---|
| **Detection Method** | Human judgement only | Fixed rule logic | Hybrid: Rule-Based + ML Model |
| **Domains Supported** | Single domain | Limited domain | Multi-domain (Finance, Healthcare, HR, Cyber, Education) |
| **Accuracy** | Subjective (~65–75%) | Moderate (~70%) | High (~85–90% ML-backed) |
| **Consistency** | Low (varies by person) | Medium | High (standardised logic + ML) |
| **Real-Time Decision** | Slow/manual | Fast | Real-time API-based decisions |
| **Human Override** | Not tracked | Not supported | Fully supported + logged |
| **Error Handling** | Manual correction | Static rules | Adaptive + ML confidence scoring |
| **Scalability** | Low | Medium | High (API + modular design) |
| **AI vs ML Comparison** | Not available | Not available | Fully supported (agreement tracking) |
| **User Interface** | Manual forms | Basic UI | Modern React dashboard + analytics |
| **Cost** | High manpower cost | Low setup cost | Low cost + scalable (~software-based) |

| | | | |
|---|---|---|---|
| **Governance & Monitoring** | None | None | Governance Score + Admin Dashboard |
| **Decision Tracking** | Not available | Partial | Full audit trail (logs + timestamps) |
| **Data Storage** | Paper/manual | Minimal logs | Structured DB (SQLite logs) |
| **Explainability** | Depends on human | Limited | XAI (feature-based explanation) |

*Table 6.2* ***Testing the New System vs Existing Systems***

## 6.2 Test Cases

*Table 6.3* **Test Cases**

| TC ID | Module | Input Description | Expected Output | Actual Output | Status |
|---|---|---|---|---|---|
| TC-01 | Finance | High income, high credit score | APPROVE | APPROVE | Pass |
| TC-02 | Finance | Low income, low credit score | REJECT | REJECT | Pass |
| TC-03 | Healthcare | High BP, low oxygen | HIGH RISK | HIGH RISK | Pass |
| TC-04 | Healthcare | Normal vitals | LOW RISK | LOW RISK | Pass |
| TC-05 | HR | High experience, strong skills | SHORTLIST | SHORTLIST | Pass |
| TC-06 | HR | Low experience, weak skills | REJECT | REJECT | Pass |

| | | | | | |
|---|---|---|---|---|---|
| TC-07 | Cybersecurity | High transaction + multiple login attempts | FRAUD | FRAUD | Pass |
| TC-08 | Cybersecurity | Normal transaction | SAFE | SAFE | Pass |
| TC-09 | Education | Low attendance + low marks | HIGH RISK | HIGH RISK | Pass |
| TC-10 | Education | Good performance | LOW RISK | LOW RISK | Pass |
| TC-11 | Logging | API call made | Entry inserted in database | Entry inserted | Pass |
| TC-12 | Admin Panel | Fetch logs | Logs displayed | Logs displayed | Pass |
| TC-13 | Override | Change AI decision | Updated human decision stored | Stored correctly | Pass |
| TC-14 | API Error | Missing input fields | Graceful error handling | No crash, handled properly | Pass |

Testing of the HDIS system was conducted using structured test cases to validate the correctness of API responses, database operations, and frontend integration across all domains. The following table summarises representative test cases used during system validation. All test cases passed successfully, confirming that the system behaves as expected under both normal and edge-case conditions.The complete HDIS system was evaluated under real usage conditions by submitting multiple inputs across all domains through the frontend interface and monitoring backend responses.The system not only meets functional requirements but also introduces governance and transparency features, making it suitable for real-world applications where accountability and accuracy are critical.

# CONCLUSION

The Human Decision Intelligence System (HDIS) successfully integrates rule-based artificial intelligence, machine learning models, explainable AI, and a full-stack web application into a unified decision-support platform. The system demonstrates the capability to provide intelligent, domain-specific decisions across Finance, Healthcare, HR, Cybersecurity, and Education, while maintaining transparency and auditability through decision logs and governance mechanisms.

A key technical contribution of the project is the hybrid decision pipeline, where deterministic rule-based logic is combined with probabilistic machine learning models to enhance reliability. The system achieves an overall macro F1-score of approximately 0.87, indicating strong predictive performance across all domains. Additionally, the AI–ML agreement rate of over 89% confirms consistency between rule-based and data-driven approaches. The implementation of a human override mechanism further strengthens the system by allowing manual intervention in critical scenarios, ensuring accountability and continuous improvement. The governance score module provides a quantitative measure of system reliability, enabling monitoring of override rates and decision alignment.

From a system design perspective, the project utilises a lightweight and efficient architecture consisting of a Flask backend, React frontend, SQLite database, and scikit-learn models. The average API response time of under 500 ms ensures real-time usability, while the modular design allows easy scalability and extension to additional domains. Beyond technical performance, the system's most significant contribution lies in its practical applicability. By presenting complex AI decisions through an intuitive user interface and an admin dashboard, the platform bridges the gap between advanced machine learning techniques and real-world decision-making. It enables users to not only receive predictions but also understand, verify, and override them when necessary.

In conclusion, the HDIS project demonstrates a robust, scalable, and interpretable approach to intelligent decision systems. It highlights the importance of combining AI, ML, and human judgment to build trustworthy systems suitable for real-world deployment across multiple industries.

# FUTURE ENHANCEMENTS

While the current Human Decision Intelligence System (HDIS) satisfies all functional and performance requirements and demonstrates strong results across multiple domains, several enhancements can further improve its scalability, intelligence, and real-world applicability:

### Advanced Machine Learning Models

The current implementation uses Logistic Regression for its simplicity and interpretability. Future versions of the system can incorporate more advanced models such as Random Forests, Gradient Boosting (XGBoost), or Deep Neural Networks to capture complex, non-linear relationships in decision data. These models can significantly improve prediction accuracy, especially in domains like Healthcare and Cybersecurity where patterns are more complex.

### Adaptive Learning and Continuous Model Updates

Currently, models are trained offline using historical data. A future enhancement involves implementing continuous learning pipelines, where models are periodically retrained using new decision logs collected from real users.Automated retraining pipelines using scheduled jobs or cloud services can ensure that models remain up-to-date without manual intervention.

### Real-Time Analytics and Visualization

The admin dashboard can be enhanced with real-time analytics using interactive charts and visualizations. Features such as: Live decision streams, Domain-wise performance graphs, Override trend analysis, AI vs ML divergence tracking can provide deeper insights into system behaviour and performance.

### Integration with External Systems

The HDIS platform can be integrated with external enterprise systems such as: Banking systems (for loan approval automation), Hospital management systems (for risk prediction), HR platforms (for candidate evaluation), Cybersecurity monitoring tools

This integration would transform the system into a deployable enterprise solution.

# REFERENCES


1. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.This paper introduces the scikit-learn library, which provides efficient tools for machine learning. It was used in this project for implementing Logistic Regression models and evaluation metrics.
2. Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). Applied Logistic Regression (3rd ed.). Wiley.This book explains the theoretical foundation of logistic regression used for binary classification. It supports the model design used across multiple domains in the system.
3. Molnar, C. (2020). Interpretable Machine Learning. Lulu.com.This work focuses on explainable AI techniques. It guided the implementation of explainability features (XAI) in the system for understanding model decisions.
4. Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why should I trust you?" Explaining the predictions of any classifier. In Proceedings of the ACM SIGKDD Conference.This paper introduces LIME, a method for explaining machine learning predictions. It inspired the explainable decision module in the project.
5. Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions (SHAP). In Advances in Neural Information Processing Systems (NeurIPS).This paper presents SHAP values for feature importance. It provides a strong foundation for advanced explainability in AI systems.
6. Tan, C., Steinbach, M., & Kumar, V. (2019). Introduction to Data Mining (2nd ed.). Pearson.This book covers fundamental data mining concepts such as classification and feature engineering, which are applied in this system.
7. [Han, J., Kamber, M., & Pei, J. (2011). Data Mining: Concepts and Techniques (3rd ed.). Morgan Kaufmann.This reference provides core concepts of data preprocessing and model evaluation, used during dataset preparation and model training.
8. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.This book explains modern machine learning and neural network concepts, forming the theoretical background for AI-based decision systems.

9. Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., et al. (2015). Hidden technical debt in machine learning systems. In Advances in Neural Information Processing Systems.This paper highlights challenges in deploying ML systems in production. It influenced the system design to ensure maintainability and scalability.
10. Breiman, L. (2001). Random forests. This paper introduces ensemble learning methods, which are considered for future enhancements of the system.
11. Bishop, C. M. (2006). Pattern Recognition and Machine Learning. A foundational text covering probabilistic models and classification techniques relevant to this project.
12. Dietterich, T. G. (2000). Ensemble methods in machine learning.This paper explains combining multiple models to improve performance, which is proposed in future enhancements.
13. Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach.A comprehensive reference for AI concepts including decision-making systems and intelligent agents.
14. Provost, F., & Fawcett, T. (2013). Data Science for Business.This book explains how data-driven decision systems are applied in real-world business scenarios, similar to this project.
15. OECD (2021). AI Principles for Responsible AI Systems. This report outlines ethical and governance principles for AI systems, supporting the governance module of the project.
16. OECD (2021). AI Principles for Responsible AI Systems.This report outlines ethical and governance principles for AI systems, supporting the governance module of the project.
17. Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library.This paper introduces PyTorch, a widely used deep learning framework. It provides insights into scalable model development and influenced the design considerations for future enhancements of the system.

# BIBLIOGRAPHY

**Flask Documentation — Web Framework for Python**

https://flask.palletsprojects.com/

Accessed April 2026.

**React Documentation — Frontend Library**

https://react.dev/

Accessed April 2026.

**SQLite Documentation — Lightweight Database Engine**

https://www.sqlite.org/docs.html

Accessed April 2026.

**scikit-learn Documentation — Machine Learning Library**

https://scikit-learn.org/stable/

Accessed April 2026.

**Pandas Documentation — Data Processing Library**

https://pandas.pydata.org/docs/

Accessed April 2026.

**NumPy Documentation — Numerical Computing**

https://numpy.org/doc/

Accessed April 2026.

**Framer Motion — Animation Library for React**

https://www.framer.com/motion/

Accessed April 2026.

**Recharts — Charting Library for React**

https://recharts.org/

Accessed April 2026.

**JWT (JSON Web Token) Documentation**

https://jwt.io/introduction

Accessed April 2026.